

Linguaggi context-free

G. Michele Pinna

Anno Accademico 2021/2022

Cosa è un linguaggio

Prima di tutto con A^* indichiamo tutte le parole di lunghezza finita sull'alfabeto A , che non è altro che un insieme finito. Le parole non sono altro che una sequenza, eventualmente vuota, di simboli dell'alfabeto. Sia $A = \{a, b\}$, allora a è una parola, così come $abbab$. Con ϵ indichiamo la parola vuota, quella cioè in cui non c'è alcun simbolo dell'alfabeto. Date due parole $w, w' \in A^*$, con $w \cdot w'$ indichiamo la *concatenazione* delle due parole, che significa che scriviamo la parola w' alla destra della parola w , formando così una nuova parola. Con $|w|$ indichiamo la lunghezza di una parola, che è definita nel seguente modo: $|\epsilon| = 0$ e $|a \cdot w| = |w| + 1$. Sia data una parola $w \in A^*$, un *prefisso* di w è una parola $w' \in A^*$ tale che esiste una parola $w'' \in A^*$ e $w = w' \cdot w''$. Un *suffisso* di una parola $w \in A^*$ è una parola $w'' \in A^*$ tale che esiste una parola $w' \in A^*$ e $w = w' \cdot w''$. Un *infixo* di una parola $w \in A^*$ è una parola $\bar{w} \in A^*$ tale che esistono due parole $w', w'' \in A^*$ e $w = w' \cdot \bar{w} \cdot w''$.

Possiamo ora ricordare cosa è un linguaggio su un alfabeto A .

Definizione 1 Sia A un alfabeto. Un linguaggio su A è un sottoinsieme di A^* .

Consideriamo $A = \{a, b\}$, allora un linguaggio può essere l'insieme delle parole di lunghezza pari in cui compaiono il doppio di a rispetto alle b . Alcune parole di questo linguaggio sono $aaaabb$, $abbaaaa$, $baabaa$ e così via. È bene ricordare che un linguaggio può contenere infinite parole, tutte di lunghezza finita. In generale considereremo solo linguaggi infiniti (che contengono un numero infinito di parole) dato che i linguaggi finiti sono certamente dei linguaggi regolari.

Grammatiche

Diamone prima una definizione informale: una grammatica è sostanzialmente un insieme di *regole* che descrivono o prescrivono la formazione di delle parole di un linguaggio, che, dal punto di vista della programmazione, sono le frasi sintatticamente corrette di un linguaggio di programmazione. Quindi, per noi, una grammatica non è altro che un insieme di regole per *generare* le parole del linguaggio.

Prima di tutto formalizziamo la nozione di *produzione*.

Definizione 2 Sia A un alfabeto. Una produzione su A è una coppia (α, β) tale che

1. $\alpha, \beta \in A^*$, e
2. $\alpha \neq \epsilon$.

L'idea che sta dietro a questa nozione è banale. Devo *riscrivere* il lato sinistro della produzione (α) con in lato destro (β), all'interno di una parola dove compare, come *infisso*, il lato sinistro della produzione. La seconda condizione impone che non si possa riscrivere la parola vuota, come sarà chiaro nel seguito. La produzione (α, β) viene normalmente scritta come $\alpha \rightarrow \beta$.

Osserviamo che le produzioni su A non sono altro che una relazione su $A^* \times A^*$. Quelle che considereremo saranno in più tali che $|\alpha| = 1$, cioè si riscrive un solo simbolo.

Definizione 3 Una grammatica su Σ è la quadrupla $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ dove

1. $\mathbf{V}$ è un insieme finito di simboli non terminali, e Σ è un insieme finito di simboli terminali tali che $\mathbf{V} \cap \Sigma = \emptyset$,
2. $\mathcal{P}$ è un insieme di produzioni su $\mathbf{V} \cup \Sigma$ tali che il primo elemento della coppia appartiene sempre a $\mathbf{V}$, quindi $\mathcal{P} \subseteq (\mathbf{V} \cup \Sigma)^* \times (\mathbf{V} \cup \Sigma)^*$ tale che per ogni $\alpha \rightarrow \beta \in \mathcal{P}$ abbiamo che $|\alpha| = 1$ e $\alpha \in \mathbf{V}$, e
3. $S \in \mathbf{V}$ è il simbolo iniziale.

Prima di formalizzare cosa è una derivazione in una grammatica, diamone un esempio. Consideriamo la grammatica $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S0, S \rightarrow 1S1, S \rightarrow \epsilon\}, S)$, come la usiamo? l'idea è di *derivare* parole su Σ^* applicando le regole definite dalle produzioni, partendo dal simbolo iniziale S . Supponiamo di voler derivare la parola 011110. Dato che le produzioni delle grammatiche che stiamo considerando riscrivono solo simboli non terminali, dobbiamo trovare all'interno della parola che stiamo riscrivendo il simbolo non terminale che è il lato sinistro di una qualche produzione. Quindi da S deriviamo $0S0$ usando la produzione $S \rightarrow 0S0$, poi, da questa, deriviamo $01S10$ usando la produzione $S \rightarrow 1S1$, poi deriviamo $011S110$ usando la medesima produzione $S \rightarrow 1S1$ ed infine deriviamo 011110 usando la produzione $S \rightarrow \epsilon$.

Data una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$, un passo di derivazione è definito come segue:

Definizione 4 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica e $\alpha \in (\mathbf{V} \cup \Sigma)^+$. Un passo di derivazione $\alpha \Rightarrow \gamma$ usando una produzione $V_i \rightarrow \alpha_i$ è definito come segue:

1. $\alpha = \beta V_i \delta$, con $\beta, \delta \in (\mathbf{V} \cup \Sigma)^*$, e
2. $\gamma = \beta \alpha_i \delta$.

Una derivazione non è altro che la chiusura riflessiva e transitiva della relazione $\Rightarrow$:

Definizione 5 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica e $\alpha \in (\mathbf{V} \cup \Sigma)^+$. Deriviamo da $\alpha \in (\mathbf{V} \cup \Sigma)^+$ la parola $\beta \in (\mathbf{V} \cup \Sigma)^*$ se e solo se $\alpha \xRightarrow{*} \beta$.

Per indicare che deriviamo β da α scriveremo $\alpha \xRightarrow{*} \beta$. Chiaramente questa definizione implica che esiste una sequenza finita di passi che consentono di arrivare da α a β con passi di derivazione successivi.

Vediamo un altro esempio: data la grammatica $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1, S \rightarrow \epsilon\}, S)$, deriviamo a partire da S :

$$\begin{array}{ll} S \Rightarrow 0S1 & \text{usando } S \rightarrow 0S1 \\ 0S1 \Rightarrow 00S11 & \text{usando } S \rightarrow 0S1 \\ 00S11 \Rightarrow 000S111 & \text{usando } S \rightarrow 0S1 \\ 000S111 \Rightarrow 000111 & \text{usando } S \rightarrow \epsilon \end{array}$$

Osserviamo che abbiamo derivato la parola 0^31^3 , e possiamo immaginare che il linguaggio generato da questa grammatica sia il linguaggio di tutte le parole tali che contengono tanti 0 quanti 1 e tutti gli 0 stanno a sinistra degli 1.

Possiamo ora formalizzare quale è il linguaggio generato da una grammatica G .

Definizione 6 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. Il linguaggio generato da G è l'insieme delle parole fatte da soli simboli terminali derivabili da S . Quindi $\mathcal{L}(G) = \{w \in \Sigma^* \mid S \xRightarrow{*} w\}$.

Spesso diremo, data una grammatica G , che questo è il linguaggio di G .

Sia $G = (\{S\}, \{0, 1\}, \{S \rightarrow 0S1, S \rightarrow \epsilon\}, S)$ la grammatica. Vediamo che $\mathcal{L}(G) = \{0^i1^i \mid i \geq 0\}$.

Per induzione sulla lunghezza della derivazione mostriamo che $S \xRightarrow{n} w$ implica che o w ha un prefisso di 0^n seguito da S , seguito da un suffisso 1^n , oppure $w = 0^{n-1}1^{n-1}$.

(BASE)

$S \Rightarrow \epsilon$ e $S \Rightarrow 0S1$ soddisfano la proprietà

(INDUZIONE)

Supponiamo valga per n , quindi $S \xRightarrow{n} w$ e $w = 0^{n-1}1^{n-1}$ oppure $w = 0^nS1^n$.

Consideriamo $S \xRightarrow{n+1} w'$. Abbiamo ancora due casi: $S \xRightarrow{n} w = 0^{n-1}1^{n-1} \xRightarrow{*} 0^{n-1}1^{n-1} = w'$, oppure $S \xRightarrow{n+1} w = 0^{n-1}S1^{n-1}$ e allora o $0^{n-1}S1^{n-1} \Rightarrow 0^{n-1}0S11^{n-1}$ usando la $S \rightarrow 0S1$ oppure $0^{n-1}S1^{n-1} \Rightarrow 0^{n-1}011^{n-1}$ usando la $S \rightarrow \epsilon$.

In entrambi i casi abbiamo la tesi.

Da quanto visto possiamo concludere che $\mathcal{L}(G)$ è proprio $\{0^i1^i \mid i \geq 0\}$

Alberi Sintattici: Abbiamo formalizzato cosa è un passo di derivazione, e tramite questa nozione abbiamo anche stabilito cosa è un linguaggio. Vediamo ora una caratterizzazione alternativa alla nozione di derivazione, che sono gli *alberi sintattici*. Ricordiamo che un albero è un grafo etichettato con un vertice designato, detto *radice* e tale che ogni vertice è raggiungibile dalla radice con un unico cammino.

Definizione 7 Un grafo etichettato su un insieme di etichette $\mathcal{E}$ è la quadrupla (V, A, v_{in}, ℓ) dove

- X è un insieme di vertici,
- $A \subseteq X \times X$ è un insieme di archi,
- $v_{in} \in V$ è un vertice detto radice, e
- $\ell : V \rightarrow \mathcal{E}$ è una funzione totale di etichettatura.

Definiamo cosa è un cammino in un albero:

Definizione 8 Sia (V, A, v_{in}, ℓ) un grafo, un cammino ρ su tale grafo è una sequenza di vertici $v_0 \dots v_n$ tale che $\forall 1 \leq i \leq n (v_{i-1}, v_i) \in A$. Del cammino ρ diciamo che parte da v_0 ed arriva a v .

Possiamo ora dare la nozione di albero:

Definizione 9 Sia (V, A, v_{in}, ℓ) un grafo, allora (V, A, v_{in}, ℓ) è un albero se e solo se per tutti i vertici $v \in V$ esiste un unico cammino da v_{in} a v .

Di un albero diciamo che un vertice v è un nodo interno se esiste un vertice v' tale che $(v, v') \in A$, mentre v è una *foglia* se non è un nodo interno. Dato un nodo interno v , allora chiamiamo v' nodo *figlio*. Dato un vertice v , i suoi figli sono l'insieme $\{v' \mid (v, v') \in A\}$, e questo insieme è indicato con $\text{succ}(v)$. Osserviamo che una foglia v è tale che $\text{succ}(v) = \emptyset$. Per noi sarà importante sottolineare che per ogni nodo interno (che quindi ha dei figli) l'ordine con cui sono elencati i figli (scritti da sinistra verso destra) è rilevante.

Possiamo formalizzare la nozione di albero sintattico.

Definizione 10 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. Un albero sintattico di G è un albero (V, A, v_{in}, ℓ) etichettato su $\mathbf{V} \cup \Sigma \cup \{\epsilon\}$ tale che

- esiste una relazione $\prec \subseteq V \times V$,
- ogni vertice (interno o foglia) ha un'etichetta in $\mathbf{V} \cup \Sigma \cup \{\epsilon\}$,
- ogni vertice interno è etichettato con una variabile in $\mathbf{V}$,
- ogni foglia è etichettata con un simbolo da $\mathbf{V} \cup \Sigma \cup \{\epsilon\}$,

- se un vertice interno v è etichettato con X ed i suoi figli $\text{succ}(v) = \{v_1, \dots, v_n\}$ sono etichettati con $X_1, X_2, \dots, X_n$ allora
 - $\text{succ}(v) \neq \emptyset$,
 - $\prec^* \cap (\text{succ}(v) \times \text{succ}(v))$ è un ordine totale, e
 - c'è una produzione $X \rightarrow X_1 X_2 \dots X_n$ tale che $\ell(v_i) = X_i$ e $\forall i \leq j \leq n$ $v_i \prec^* v_j$.

Perché abbiamo introdotto la nozione di albero sintattico? L'idea è che se leggiamo le foglie da sinistra a destra possiamo trovare una stringa derivabile dal simbolo non terminale che è nella radice.

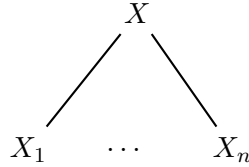
Alberi sintattici e derivazioni Collegiamo gli alberi sintattici alle derivazioni

Teorema 1 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$. Allora w è derivabile da $X \in \mathbf{V}$ se e solo se esiste un albero sintattico in cui leggiamo w e che ha come etichetta del nodo radice X .

(SOLO SE)

Per induzione sulla lunghezza della derivazione.

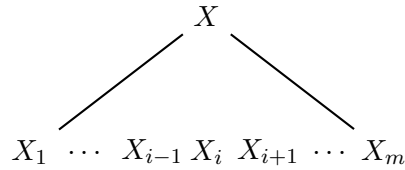
Se la derivazione è di un solo passo allora abbiamo l'albero



con $w = X_1 \dots X_n$ è ottenuta dalla regola $X \rightarrow X_1 \dots X_n$, con $X_i \in \mathbf{V} \cup \Sigma$

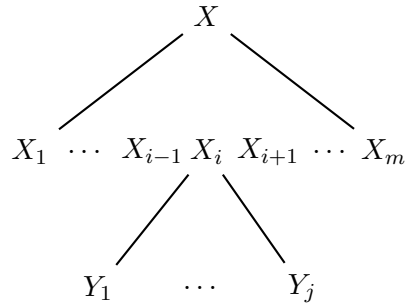
Supponiamo di aver mostrato che vale per derivazioni lunghe n , mostriamo che vale per $n + 1$.

Per ogni derivazione lunga n abbiamo un albero:



in cui il cammino più lungo è n , che corrisponde alla derivazione da X a $X_1 \dots X_m \in (\mathbf{V} \cup \Sigma)^*$, quindi $X \xRightarrow{n} X_1 \dots X_{i-1} X_i X_{i+1} \dots X_m$.

Per avere una derivazione lunga $n + 1$ dobbiamo avere che un X_i è una variabile, ed esiste una produzione $X_i \rightarrow Y_1 \dots Y_j$, con $Y_1, \dots, Y_j \in \mathbf{V} \cup \Sigma$, e abbiamo $X \xRightarrow{n} X_1 \dots X_{i-1} X_i X_{i+1} \dots X_m \Rightarrow X_1 \dots X_{i-1} Y_1 \dots Y_j X_{i+1} \dots X_m$ e a questa derivazione corrisponde l'albero

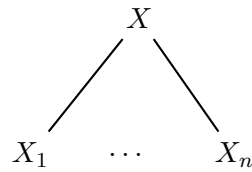


in cui, se leggiamo le foglie da sinistra a destra, ritroviamo esattamente la parola derivata $X_1 \cdots X_{i-1} Y_1 \cdots Y_j X_{i+1} \cdots X_m$.

(SE)

Per induzione sulla lunghezza del cammino più lungo nell'albero sintattico.

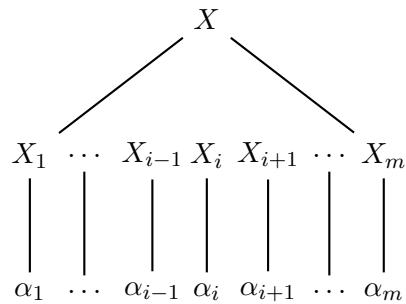
Se la lunghezza del cammino più lungo è 1 allora abbiamo l'albero



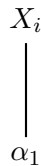
ed essendo questo un albero sintattico abbiamo la tesi dato che esiste una produzione $X \rightarrow X_1 \cdots X_n$, e quindi della derivazione $X \Rightarrow X_1 \cdots X_n$.

Supponiamo valga per alberi in cui il cammino di lunghezza massima è lungo n .

In un albero in cui il cammino più lungo ha lunghezza $n + 1$ abbiamo la seguente situazione:



ed in questa ogni sottoalbero



ha profondità minore od uguale ad n , quindi $X_i \xRightarrow{*} \alpha_i$, facendo l'assunzione che se l'etichetta di X_i è un terminale allora il vertice è una foglia, se invece se l'etichetta di X_i è un non terminale, allora X_i si deriva α_i con una derivazione lunga n , .

Quindi abbiamo che $X \Rightarrow X_1 \cdots X_{i-1} X_i X_{i+1} \cdots X_m \xRightarrow{*} \alpha_i \cdots X_{i-1} X_i X_{i+1} \cdots X_m \xRightarrow{*} \cdots \xRightarrow{*} \alpha_1 \cdots \alpha_{i-1} \alpha_i \alpha_{i+1} \cdots \alpha_m$ è la derivazione cercata.

Avendo mostrato che sia l'albero sintattico che le derivazioni sono equivalenti abbiamo che, data una grammatica, ci sono vari modi per ottenere le parole appartenenti al linguaggio generato dalla grammatica:

1. costruire una derivazione
2. costruire un albero sintattico

I due metodi, abbiamo visto, sono equivalenti.

Ne esiste un terzo: *inferenza induttiva*. L'idea è che ogni non terminale della grammatica genera un certo numero di parole. La parola che cerchiamo di derivare è la concatenazione di opportune parole. In effetti ricade nelle due precedenti.... Si dimostra facilmente che questo metodo è equivalente ai due precedenti.

Gerarchia di linguaggi

Introduciamo le grammatiche regolari.

Definizione 11 Una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ è detta regolare se e solo se tutte le sue produzioni sono della forma $A \rightarrow aB$, con $a \in \Sigma$ e $B \in \mathbf{V}$, oppure $A \rightarrow a$, con $a \in \Sigma \cup \{\epsilon\}$, e se $A \rightarrow \epsilon$ allora $A = S$.

Teorema 2 Un linguaggio è regolare se e solo se è generato da una grammatica regolare.

(SOLO SE)

Basta associare ad un DFA una grammatica regolare.

Prendiamo $A = (Q, \Sigma, \delta, q_0, F)$, costruiamo la grammatica $G = (Q, \Sigma, \mathcal{P}, q_0)$ dove le produzioni sono $q \rightarrow aq'$ se $\delta(q, a) = q'$ e aggiungiamo le produzioni $q \rightarrow a$ se $\delta(q, a) \in F$ e la produzione $q_0 \rightarrow \epsilon$ se $q_0 \in F$.

Ovviamente $w \in \mathcal{L}(A)$ sse $\hat{\delta}(q_0, w) \in F$ ma questo equivale chiaramente a $q_0 \xRightarrow{*} w$ sse $w \in \mathcal{L}(G)$ (osserviamo che se $\epsilon \in \mathcal{L}(A)$ allora $q_0 \in F$)

(SE)

Abbiamo una grammatica regolare $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$, costruiamo un automa a stati finiti.

Prendiamo $A = (\mathbf{V} \cup \{q_f\}, \Sigma, \delta, S, \{q_f\})$, dove δ è definita in modo che $B \in \delta(A, a)$ se $A \rightarrow aB$ è una produzione di $\mathcal{P}$ $q_f \in \delta(A, a)$ se $A \rightarrow a$ è una produzione di $\mathcal{P}$.

Come prima $w \in \mathcal{L}(G)$ sse $S \xRightarrow{*} w$ ma questo equivale a dire $\hat{\delta}(S, w) \cap F \neq \emptyset$ sse $w \in \mathcal{L}(A)$

Corollario 1 *Ogni linguaggio regolare è anche context free.*

Un linguaggio regolare viene generato da una grammatica con opportune restrizioni (viene chiamata lineare destra). Abbiamo percorso il primo passo della gerarchia di Chomsky: dai linguaggi regolari a quelli context free.

0.1 I linguaggi context free e quelli regolari

Abbiamo definito una classe di linguaggi, quelli regolari, e abbiamo introdotto un modo per generare parole (la grammatica). Diamo la seguente definizione:

Definizione 12 *Sia fissato un alfabeto Σ . La classe dei linguaggi context free su Σ è formata da tutti i linguaggi $\mathcal{L} \subseteq \Sigma^*$ per cui esiste una grammatica G tale che $\mathcal{L}(G) = \mathcal{L}$. La indichiamo come L_{cf} .*

Indichiamo con L_{reg} la classe dei linguaggi regolari, che conosciamo bene.

Teorema 3 $L_{reg} \subset L_{cf}$.

Chiaramente $L_{reg} \subseteq L_{cf}$ dato che abbiamo visto che un linguaggio regolare viene generato da una grammatica regolare. Per il contenimento proprio osserviamo che $\{0^n 1^n \mid n \geq 1\}$ sta in L_{cf} ma non in L_{reg}

Derivazioni left-most e right-most

Definizione 13 *Data la grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ e la derivazione $\alpha \xRightarrow{*} \beta$. Diciamo che la derivazione è leftmost se e solo se sono stati riscritti i non terminali più a sinistra. In questo caso indichiamo $\alpha \xRightarrow{*}_{lm} \beta$*

Analogamente:

Definizione 14 *Data la grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ e la derivazione $\alpha \xRightarrow{*} \beta$. Diciamo che la derivazione è rightmost se e solo se sono stati riscritti i non terminali più a destra. In questo caso indichiamo $\alpha \xRightarrow{*}_{rm} \beta$*

Vediamo un esempio. Consideriamo la grammatica $G = (\{E\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow \mathbf{a}, E \rightarrow \mathbf{b}, E \rightarrow \emptyset, E \rightarrow \epsilon, E \rightarrow (E), E \rightarrow E^*, E \rightarrow E + E, E \rightarrow E \cdot E\}, E)$.

Deriviamo $\mathbf{a} + (\mathbf{b}^* + \mathbf{a})$ in due modi:

(leftmost)

$$\begin{aligned} E &\Rightarrow E + E \Rightarrow \mathbf{a} + E \Rightarrow \mathbf{a} + (E) \Rightarrow \mathbf{a} + (E + E) \Rightarrow \mathbf{a} + (E^* + E) \Rightarrow \\ &\mathbf{a} + (\mathbf{b}^* + E) \Rightarrow \mathbf{a} + (\mathbf{b}^* + \mathbf{a}) \end{aligned}$$

(rightmost)

$$\begin{aligned} E &\Rightarrow E + E \Rightarrow E + (E) \Rightarrow E + (E + E) \Rightarrow E + (E + \mathbf{a}) \Rightarrow E + (E^* + \mathbf{a}) \Rightarrow \\ &E + (\mathbf{b}^* + \mathbf{a}) \Rightarrow \mathbf{a} + (\mathbf{b}^* + \mathbf{a}) \end{aligned}$$

Ambiguità: Consideriamo la grammatica

$$G = (\{E\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid (E) \mid E^* \mid E + E, \mid E \cdot E\}, E)$$

e la parola $\mathbf{a} + \mathbf{a} \cdot \mathbf{b}$ ha due derivazioni leftmost:

1. $E \Rightarrow E + E \Rightarrow \mathbf{a} + E \Rightarrow \mathbf{a} + E \cdot E \Rightarrow \mathbf{a} + \mathbf{a} \cdot E \Rightarrow \mathbf{a} + \mathbf{a} \cdot \mathbf{b}$
2. $E \Rightarrow E \cdot E \Rightarrow E + E \cdot E \Rightarrow \mathbf{a} + E \cdot E \Rightarrow \mathbf{a} + \mathbf{a} \cdot E \Rightarrow \mathbf{a} + \mathbf{a} \cdot \mathbf{b}$

Definizione 15 Una grammatica G è ambigua se per una stessa parola esistono due derivazioni leftmost diverse.

Diverse significa che gli alberi sintattici associati non sono isomorfi.

Ambiguità eliminabili: Consideriamo la grammatica

$$G = (\{E\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid (E) \mid E^* \mid E + E, \mid E \cdot E\}, E)$$

sappiamo che gli operatori hanno precedenze e legano più di altri, o godono di proprietà... possiamo tenerne conto.

Sappiamo che il ‘ $\cdot$ ’ lega di più del ‘ $+$ ’, ed il ‘ $*$ ’ più del ‘ $\cdot$ ’: teniamone conto cercando di *non confondere* le derivazioni.

Introduciamo dei nuovi simboli non terminali:

- $E \rightarrow T \mid E + T$ - un’espressione regolare è la somma di due espressioni generate da T ed E oppure da una sola in cui *all’esterno* non c’è un $+$,
- $T \rightarrow F \mid T \cdot F$ - un’espressione regolare è la concatenazione di due espressioni generate da T ed F oppure da una sola in cui *all’esterno* non c’è un $*$,
- $F \rightarrow K \mid F^*$ - un’espressione regolare ha come operatore principale lo star di Kleene,
- $K \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon$ - Il resto... tengo anche conto che tutte le operazioni associative in effetti associano a sinistra.

Deriviamo $\mathbf{a} + \mathbf{a} \cdot \mathbf{b}$

$$E \Rightarrow E + T \Rightarrow \mathbf{a} + T \Rightarrow \mathbf{a} + T \cdot F \Rightarrow \mathbf{a} + F \cdot F \Rightarrow \mathbf{a} + K \cdot F \Rightarrow \mathbf{a} + \mathbf{a} \cdot F \Rightarrow \mathbf{a} + \mathbf{a} \cdot K \Rightarrow \mathbf{a} + \mathbf{a} \cdot \mathbf{b}$$

Se provassimo a derivare dal ‘ $\cdot$ ’ avremmo

$$E \Rightarrow T \Rightarrow T \cdot F \Rightarrow F \cdot F \Rightarrow K \cdot F$$

ma da qui o metto parentesi oppure un terminale.

In effetti ntroduciamo nuove variabili per *forzare* il corretto parsing dell’espressione.

Ambiguità eliminabili e no: Possiamo eliminare qualsiasi ambiguità, ovvero esiste per ogni linguaggio generato da una grammatica ambigua, una grammatica che genera lo stesso linguaggio e che non è ambigua?

Consideriamo $\mathcal{L} = \{a^n b^n c^m d^m \mid n, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n, m \geq 1\}$

A parte le stringhe $a^n b^n c^n d^n$ che appartengono ad entrambi i linguaggi, devo decidere da subito se voglio generare una stringa del primo linguaggio o una del secondo...

Mostriamo che $\mathcal{L}$ è context free (i.e., forniamo una grammatica).

$S \rightarrow AB \mid aEd$ da S derivo AB (per il primo) o aEd (per il secondo)

$\left. \begin{array}{l} A \rightarrow aAb \mid ab \\ B \rightarrow cBd \mid cd \end{array} \right\}$ per generare $\{a^n b^n c^m d^m \mid n, m \geq 1\}$

$\left. \begin{array}{l} E \rightarrow aEd \mid F \\ F \rightarrow bFc \mid bc \end{array} \right\}$ per generare $\{a^n b^m c^m d^n \mid n, m \geq 1\}$

La dimostrazione che non esiste una grammatica non ambigua è complessa e la omettiamo. Importante è osservare che esistono linguaggi context free che sono inerentemente ambigui.

Esercizi

1. Scrivere una grammatica che genera $\{a^i b^j \mid i \leq j\}$
2. Scrivere una grammatica che genera $\{a^i b^j c^k \mid i \neq j \text{ oppure } j \neq k\}$
3. Scrivere una grammatica che genera $\{w \in \{0, 1\}^* \mid w \text{ interpretato come numero binario è divisibile per } 8\}$
4. Mostrare che la grammatica $G = (\{E\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow \mathbf{a}, E \rightarrow \mathbf{b}, E \rightarrow \emptyset, E \rightarrow \epsilon, E \rightarrow (E), E \rightarrow E^*, E \rightarrow E + E, E \rightarrow E \cdot E\}, E)$ genera tutte e sole le espressioni regolari
5. Scrivere una grammatica che genera tutte le parole palindrome (cioè tali che $w = w^R$) su $\{0, 1\}$
6. Scrivere una grammatica che genera tutte le parole su $\{0, 1\}$ che hanno lo stesso numero di 0 e di 1
7. Sia G una grammatica in cui nessuna produzione è della forma $A \rightarrow \epsilon$. Mostrare che se $w \in \mathcal{L}(G)$, $|w| = n$ e ha una derivazione di m passi, allora w ha un albero sintattico di $n + m$ nodi
8. considerate la grammatica $S \rightarrow 0S \mid 1S \mid 0A \mid 1A, A \rightarrow 0B, B \rightarrow 0C, C \rightarrow 0$, trovate gli alberi sintattici di 001000 e 11000
9. Scrivete una grammatica che genera tutte le parole su $\{(\cdot, [\cdot, \{\cdot, \cdot\}, \cdot], \cdot)\}$ tali, per ogni posizione all'interno di una parola del linguaggio, il numero di parentesi di un

tipo aperte è sempre maggiore o uguale al numero di parentesi chiuse, e tale che il numero di parentesi dello stesso tipo è sempre perfettamente bilanciato (e.g. $(\llbracket((\{\})\{\})\{\})\{\})()$ appartiene al linguaggio ma $((\{\{ e \})$ no)

10. considerate $S \rightarrow aS \mid aSbS \mid \epsilon$. Mostrate che è ambigua (costruite due derivazioni sinistre per aab)
11. trovate una grammatica non ambigua per il linguaggio dell'esercizio precedente.

Dispositivi per riconoscere linguaggi

Abbiamo bisogno di un dispositivo per riconoscere le parole di un linguaggio context-free. Sappiamo che non ci basta un automa a stati finiti per riconoscere un linguaggio come $\{0^n 1^n \mid n \geq 0\}$, che è context-free. Quale è la *debolezza* degli automi a stati finiti? Hanno una *memoria* limitata (dal numero degli stati). Per riconoscere $0^n 1^n$ devo ricordarmi quanti 0 ho visto, quindi non posso codificare tale memoria con gli stati. Quale è la memoria più semplice che abbiamo a disposizione? La *pila*.

Diamo una descrizione informale di un automa con una memoria a pile: ho un dispositivo di controllo (un automa a stati finiti) e una memoria a pila. Leggo sul nastro d'ingresso un simbolo (che può essere anche la parola vuota) e *leggo* sulla pila il simbolo che c'è in cima. A seconda dello stato si:

- leva il simbolo dalla cima della pila e lo si sostituisce con una parola
- si consuma il simbolo d'input (tranne che se si è letta la parola vuota)
- si cambia eventualmente stato

Definizione 16 Un automa a pila (o PDA, Push-Down Automaton) è la 7-upla $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ dove

1. Q è un insieme finito di stati;
2. Σ è l'alfabeto d'input, ed è finito;
3. Γ è l'alfabeto della pila, ed è finito;
4. $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$ è la funzione di transizione
5. q_0 è lo stato iniziale;
6. Z_0 è il simbolo iniziale che è presente sulla pila all'inizio della computazione
7. $F \subseteq Q$ è l'insieme degli stati accettanti o finali.

Non è necessario che Σ e Γ siano disgiunti, ed il PDA è non deterministico: vedremo che qui il non determinismo conta! Se nei DFA la configurazione era una coppia *stato e simbolo da leggere*, ora è più complessa:

stato e simbolo da leggere (anche ϵ) e pila

Vediamo un esempio:

$$P = (\{q_0, q, q_f\}, \{0, 1\}, \{Z_0, 0\}, \delta, q_0, Z_0, \{q_f\})$$

Le mosse sono:

- $\delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$. Se leggo uno 0 e sulla pila c'è Z_0 , lo consumo e memorizzo sulla pila che ho trovato il primo zero
- $\delta(q_0, 0, 0) = \{(q_0, 00)\}$. Se leggo uno 0 e sulla pila c'è 0 (significa che ho già trovato almeno uno 0), lo consumo e memorizzo sulla pila che ho trovato un altro zero
- $\delta(q_0, 1, Z_0) = \emptyset$. Se leggo uno 1 e sulla pila c'è Z_0 , vuol dire che la parola inizia per 1
- $\delta(q_0, 1, 0) = (q, \epsilon)$. Se sono nello stato q_0 , leggo un 1 e sulla pila c'è 0, vuol dire che ho visto fino ad ora solo 0 ed ora inizio a vedere 1. Levo lo 0 dalla pila....
- $\delta(q, 1, 0) = (q, \epsilon)$. Se sono nello stato q , leggo un 1 e sulla pila c'è 0, vuol dire che ho iniziato a vedere 1. Levo lo 0 dalla pila e continuo
- $\delta(q, 0, a) = \emptyset$. Se sono nello stato q e leggo uno 0 allora qualsiasi simbolo c'è sulla pila ($a \in \Gamma$), la parola non appartiene al linguaggio
- $\delta(q, \epsilon, Z_0) = (q_f, \epsilon)$, se ho consumato tutto l'input, ho messo tanti 0 sulla pila quanti ne ho tolto, allora posso accettare la parola
- le altre danno sempre l'insieme vuoto.

Formalizziamo cosa è uno stato di un PDA: diamo la nozione di *descrizione istantanea* che ci serve per descrivere ogni *snapshot* del sistema.

Definizione 17 Dato un PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, una *descrizione istantanea* è la tripla (q, w, γ) dove $q \in Q$ indica in quale stato si trova l'automa a pila P , $w \in \Sigma^*$ indica cosa ancora devo leggere e $\gamma \in \Gamma^*$ indica cosa è presente sulla pila. La *descrizione istantanea iniziale* è (q_0, w, Z_0) .

Come passo da una descrizione istantanea ad un'altra?

Definizione 18 Dato un PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ e una *descrizione istantanea* (q, w, γ) , la *descrizione* (q', w', γ') è *raggiungibile* da (q, w, γ) se esiste in $\delta(q, a, A)$ la coppia (q', α) tale che $w = aw'$ e $\gamma' = \alpha\beta$ posto che $\gamma = A\beta$.

La *transizione* viene indicata con $(q, w, \gamma) \vdash (q', w', \gamma')$.

Le descrizioni istantanee hanno delle proprietà. Abbiamo una relazione di transizione. Osserviamo che una sequenza di descrizioni istantanee assomiglia molto ad una *computazione*.

Definizione 19 Dato un PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ e una descrizione istantanea (q, w, γ) , la descrizione (q', w', γ') è raggiungibile da (q, w, γ) se esiste in $\delta(q, a, A)$ la coppia (q', α) tale che $w = aw'$ e $\gamma' = \alpha\beta$ posto che $\gamma = A\beta$. La transizione viene indicata con $(q, w, \gamma) \vdash (q', w', \gamma')$.

Facciamone la chiusura transitiva e riflessiva e indichiamola con $\vdash^*$ (mi concentro su cosa ottengo e non solo sui passi)

Teorema 4 Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un PDA e sia $(q, x, \alpha) \vdash^* (p, y, \beta)$, allora per ogni $w \in \Sigma^*$, $\gamma \in \Gamma^*$ vale che $(q, xw, \alpha\gamma) \vdash^* (p, yw, \beta\gamma)$

Per induzione sul numero di passi in $(q, x, \alpha) \vdash^* (p, y, \beta)$. Se è uno, è la definizione. Siano n e vediamo per $n + 1$. $(q, xw, \alpha\gamma) \vdash^* (p, yw, \beta\gamma)$ la riscriviamo come $(q, xw, \alpha\gamma) \vdash (q', x'w, \alpha'\gamma) \vdash^* (p, yw, \beta\gamma)$ e da questo abbiamo la tesi, visto che ritroviamo $(q', x'w, \alpha'\gamma) \vdash^* (p, yw, \beta\gamma)$ di lunghezza n .

Teorema 5 Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un PDA e $(q, xw, \alpha) \vdash^* (p, yw, \beta)$, allora anche $(q, x, \alpha) \vdash^* (p, y, \beta)$

Per analoga induzione sul numero di passi.

Se una sequenza di descrizioni istantanee (cioè una computazione) è lecita per P allora lo è anche quella sequenza in cui a tutte le ID abbiamo aggiunto lo stesso suffisso alla stringa d'input: quella parte d'input non l'ho ancora vista, e non so se la dovrò vedere. Se una sequenza di descrizioni istantanee (cioè una computazione) è lecita per P allora lo è anche quella sequenza in cui a tutte le ID abbiamo aggiunto lo stesso suffisso alla stringa che rappresenta la pila: nello stack, nella parte che non vedo, ci può essere qualsiasi cosa.

Se una sequenza di descrizioni istantanee (cioè una computazione) è lecita per P , e resta una coda d'input non consumata, allora lo è anche quella sequenza in cui a tutte le ID abbiamo levato lo stesso residuo alla stringa d'input: come l'aggiungiamo, possiamo anche levarla (attenzione, leviamo solo una parte che nella computazione non è mai stata *in ballo*).

Linguaggio accettato da un PDA: Fissiamo alcuni concetti, che giocano un ruolo essenziale:

- l'automa a pila è non deterministico (scelgo tra varie possibili mosse)
- ho una pila sulla quale, per continuare, devo leggere qualche simbolo
- ho degli stati finali o accettanti
- ho una ID iniziale

esiste un unico modo per riconoscere una parola?

- devo consumare tutto l'input
- potrei trovarmi in uno stato accettante e non mi importa molto della pila
- potrei richiedere che la pila sia vuota e non mi importa molto dello stato accettante
- cerco di capire come una ID finale possa essere...

Definizione 20 Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un PDA, allora il linguaggio riconosciuto dall'automa per stato finale è

$$\mathcal{L}_{PDA}(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \gamma) \text{ con } p \in F\}$$

Sono quindi tutte le parole che l'automa a pila riconosce perchè, consumato tutto l'input, sono in uno stato finale. Sulla pila ci possono essere moltissimi simboli.

Come ulteriore esempio, scriviamo un PDA che accetta il linguaggio $\{a^i b^j \mid i > j \text{ e } j > 0\}$.

$$(\{q_0, q, q_f\}, \{a, b\}, \{A, Z_0\}, \delta, q_0, Z_0, \{q_f\})$$

Le mosse sono:

- $\delta(q_0, a, Z_0) = \{(q_0, A)\}$ leggo una a metto un simbolo sulla pila (A) e proseguo
- $\delta(q_0, a, A) = \{(q_0, AA)\}$ leggo una a metto un simbolo sulla pila (A) e proseguo
- $\delta(q_0, b, A) = \{(q, \epsilon)\}$ leggo una b e levo il simbolo sulla pila (A) e proseguo
- $\delta(q, b, A) = \{(q, \epsilon)\}$ leggo una b e levo il simbolo sulla pila (A) e proseguo
- $\delta(q, \epsilon, A) = \{(q_f, A)\}$ non leggo nulla: dovrei aver terminato di riconoscere la parola. Se ci sono ancora simboli sulla pila allora ho contato più a che b , altrimenti no...

L'idea è che ogni volta che leggo una a metto un simbolo sulla pila, se invece leggo una b lo levo. Una computazione potrebbe essere:

$$(q_0, aaab, Z_0) \vdash (q_0, aab, A) \vdash (q_0, ab, AA) \vdash (q_0, b, AAA) \vdash (q, \epsilon, AA) \vdash (q_f, \epsilon, AA)$$

Consideriamo invece la parola $abbb$ che non appartiene al linguaggio:

$$(q_0, abbb, Z_0) \vdash (q_0, bbb, A) \vdash (q_0, bb, \epsilon)$$

e non si può proseguire.

Abbiamo lasciato intendere che ci potesse essere una definizione alternativa:

Definizione 21 Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un PDA, allora il linguaggio riconosciuto dall'automa per stack vuoto è

$$\mathcal{N}_{PDA}(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash^* (p, \epsilon, \epsilon)\}$$

Sono quindi tutte le parole che l'automa a pila riconosce perchè, consumato tutto l'input, anche sulla pila non c'è più alcun simbolo. La differenza è che non si pongono condizioni sul fatto che p possa o no essere uno stato finale, e quindi possiamo omettere gli stati accettanti: abbiamo quindi che un automa a pila con questa caratteristica sarà $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$

Vediamone un esempio. Costruiamo un PDA che accetta $\{a^i b^{2i} \mid i \geq 1\}$

$$(\{q_0, q, q_f\}, \{a, b\}, \{A, Z_0\}, \delta, q_0, Z_0)$$

- $\delta(q_0, a, Z_0) = \{(q_0, AA)\}$ leggo una a , metto AA sulla pila e proseguo
- $\delta(q_0, a, A) = \{(q_0, AAA)\}$ leggo una a , metto AAA sulla pila e proseguo
- $\delta(q_0, b, A) = \{(q, \epsilon)\}$ leggo una b e levo il simbolo sulla pila (A) e proseguo
- $\delta(q, b, A) = \{(q, \epsilon), (q_f, \epsilon)\}$ leggo una b e levo il simbolo sulla pila (A) e proseguo oppure immagino di aver terminato

Ogni volta che leggo una a metto due AA sulla pila, se invece leggo una b ne levo una. Una computazione è

$$(q_0, abb, Z_0) \vdash (q_0, bb, AA) \vdash (q, b, A) \vdash (q, \epsilon, \epsilon)$$

Accettazione per stato finale o stack vuoto?

Teorema 6 Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ un PDA che accetta per stato finale. Allora esiste $P^N = (Q \cup \{p_0, p\}, \Sigma, \Gamma \cup \{X_0\}, \delta^N, p_0, X_0)$, PDA che accetta lo stesso linguaggio per pila vuota, i.e., $\mathcal{L}_{PDA}(P) = \mathcal{N}_{PDA}(P^N)$.

Aggiungiamo uno stato p , che serve semplicemente a svuotare la pila, ed un nuovo stato iniziale p_0 . L'idea è che P^N funziona come P fino a che P consuma tutto l'input. Se è in uno stato finale allora svuota la pila. La δ^N estende la δ come segue:

- $\delta^N(p_0, \epsilon, X_0) = \{q_0, Z_0\}$ mi porto nello stato iniziale di P (meglio, nella ID iniziale)
- $\delta^N(q, \epsilon, A) = \{(p, A)\}$ per ogni $A \in \Gamma \cup \{X_0\}$ e $q \in F$ (per poter svuotare la pila)
- $\delta^N(p, \epsilon, A) = \{(p, \epsilon)\}$ per ogni $A \in \Gamma \cup \{X_0\}$ (svuoto la pila)

Ora $(p_0, w, X_0) \vdash (q_0, w, Z_0)$ usando la prima, e, supponendo che $w \in \mathcal{L}_{PDA}(P)$, abbiamo che $(q_0, w, Z_0) \vdash^* (q, \epsilon, \alpha)$, dove $q \in F$. Da (q, ϵ, α) otteniamo (p, ϵ, α) e poi possiamo svuotare la pila, quindi $(q, \epsilon, \alpha) \vdash^* (p, \epsilon, \epsilon)$. Componiamo assieme e abbiamo $(p_0, w, X_0) \vdash (q_0, w, Z_0) \vdash^* (q, \epsilon, \alpha) \vdash (p, \epsilon, \alpha) \vdash^* (p, \epsilon, \epsilon)$, quindi $\mathcal{L}_{PDA}(P) = \mathcal{N}_{PDA}(P^N)$.

Teorema 7 Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ un PDA che accetta per stack vuoto. Allora esiste $P^L = (Q \cup \{p_0, p_f\}, \Sigma, \Gamma \cup \{X_0\}, \delta^L, p_0, X_0, \{p_f\})$, PDA che accetta lo stesso linguaggio per stato finale, i.e., $\mathcal{L}_{PDA}(P^L) = \mathcal{N}_{PDA}(P)$.

- Aggiungiamo uno stato p_f accettante, nel quale vado se la pila è stata svuotata. Me ne accorgo perchè trovo il nuovo simbolo d'inizio pila. L'idea è che P^L funziona come P fino a che P consuma tutto l'input. Se la pila è vuota, vado in uno stato accettante. La δ^L estende la δ come segue:

- $\delta^L(p_0, \epsilon, X_0) = \{q_0, Z_0 X_0\}$ mi porto nello stato iniziale di P
- $\delta^L(p, \epsilon, X_0) = \{p_f, \epsilon\}$ per ogni $p \in Q$

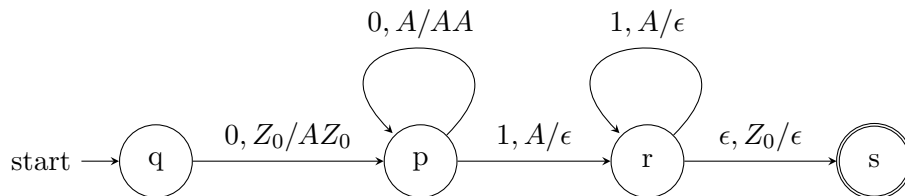
Ora $(p_0, w, X_0) \vdash (q_0, w, Z_0 X_0)$ usando la prima, e, supponendo che $w \in \mathcal{N}_{PDA}(P)$, abbiamo che $(q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon)$, dove $q \in Q$. Usando una delle proprietà delle ID, abbiamo $(q_0, w, Z_0 X_0) \vdash^* (q, \epsilon, \epsilon X_0) = (q, \epsilon, X_0)$. Da (q, ϵ, X_0) otteniamo (p_f, ϵ, X_0) . Componiamo assieme e abbiamo $(p_0, w, X_0) \vdash (q_0, w, Z_0 X_0) \vdash^* (q, \epsilon, X_0) \vdash (p_f, \epsilon, X_0)$, quindi $\mathcal{L}_{PDA}(P^L) = \mathcal{N}_{PDA}(P)$.

Abbiamo provato che la condizione d'accettazione non cambia l'espressività del dispositivo di calcolo.

Rappresentazione grafica di un PDA: Diagramma delle transizioni come per gli automi a stati finiti, solo che gli archi sono annotati in modo più complesso:

$$P = (\{q, p, r, s\}, \{0, 1\}, \{Z_0, A\}, \delta, q, Z_0, \{r\})$$

mettiamo il simbolo letto (se c'è), il simbolo in cima alla pila e quello che viene messo sulla pila al posto del simbolo...



Che dispositivo riconosce i linguaggi context free? Oramai dovrebbe essere chiaro che i PDA sono il dispositivo per riconoscere i linguaggi context free, e abbiamo già visto alcuni automi che riconoscono linguaggi che sappiamo essere context free. Ora formalizziamo la corrispondenza.

Dalle grammatiche ai PDA

Teorema 8 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. Allora esiste un PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ tale che $\mathcal{L}(G) = \mathcal{N}_{PDA}(P)$.

Definiamo il seguente automa: $P = (\{q\}, \Sigma, \mathbf{V} \cup \Sigma, \delta, q, S)$ e la δ è la seguente:

1. $\delta(q, \epsilon, A) = \{(q, \alpha) \mid A \rightarrow \alpha \text{ è una produzione in } \mathcal{P}\}$
2. $\delta(q, a, a) = \{(q, \epsilon)\}$

Vediamo prima che a ogni derivazione di parola in G corrisponde una computazione in P . Consideriamo $S \xRightarrow{*}_{lm} w$ che è $S \Rightarrow \gamma_1 \Rightarrow \gamma_2 \dots \gamma_{n_1} \Rightarrow \gamma_n = w$. Mostriamo, per induzione sulla lunghezza della derivazione di w , che $(q, w, S) \vdash^* (q, x_i, \alpha_i)$ dove $\gamma_i = \alpha_i x_i$ e il simbolo iniziale di α_i è una variabile se $\alpha_i \neq \epsilon$, per ogni i .

(BASE):

$S \xRightarrow{*} S$. Quindi $(q, \epsilon, S) \vdash^* (q, \epsilon, S)$ (usando la riflessività). Chiaramente S inizia con una variabile.

(INDUZIONE):

$S \Rightarrow \gamma_1 \Rightarrow \gamma_2 \dots \gamma_{i_1} \Rightarrow \gamma_i$. Valga per i , quindi $(q, w, S) \vdash^* (q, x_i, \alpha_i)$. Consideriamo $\gamma_i \Rightarrow \gamma_{i+1}$, dato che $\gamma_i = X \hat{\gamma}_i$ inizia per variabile, la derivazione è tale che $\gamma_{i+1} = \beta \hat{\gamma}_i$, usando la produzione $X \rightarrow \beta$. Usando la definizione di δ in 1. abbiamo che $(q, x_i, X \hat{\gamma}_i) \vdash (q, x_i, \beta \hat{\gamma}_i)$. Ora osserviamo che se β è ϵ allora dobbiamo controllare se $\hat{\gamma}_i$ inizia per variabile o no. Se si allora abbiamo la tesi: $(q, w, S) \vdash^* (q, x_i, \alpha_i) \vdash (q, x_i, \hat{\gamma}_i)$. Altrimenti leviamo, usando la definizione di δ in 2., tutti i simboli terminali, quindi $(q, x_i, \alpha_i) \vdash (q, x_i, \hat{\gamma}_i) \vdash^* (q, x_{i+1}, \gamma_{i+1})$ usando il fatto che $x_i = y_i x_{i+1}$ e $\hat{\gamma}_i = y_i \gamma_i$.

Quindi a $S \Rightarrow \gamma_1 \Rightarrow \gamma_2 \dots \gamma_{n_1} \Rightarrow \gamma_n$ corrisponde $(q, w, S) \vdash^* (q, x_i, \alpha_i)$ per ogni i , e quindi anche $(q, w, S) \vdash^* (q, \epsilon, \alpha_n)$. Osserviamo che $\alpha_n = \epsilon$ e quindi la tesi.

Mostriamo ora che se $(q, x, A) \vdash^* (q, \epsilon, \epsilon)$ allora $A \xRightarrow{*} x$, per induzione sul numero di mosse.

Una mossa: allora $(q, x, A) \vdash (q, \epsilon, \epsilon)$ ha come unica possibilità che si usi la 1. e che la produzione sia $A \rightarrow \epsilon$.

P fa n mosse: $(q, x, A) \vdash (q, x, \alpha)$ è la prima e usa $A \rightarrow \alpha$. Assumiamo $\alpha = Y_1 Y_2 \dots Y_k$, e ogni $Y_i \in \mathbf{V} \cup \Sigma$. Le successive $n-1$ mosse devono levare x dall'input e ogni Y_i dalla pila. Scomponiamo $x = x_1 x_2 \dots x_k$. Per ipotesi induttiva abbiamo che se Y_i è una variabile, vale che $(q, x_i, Y_i) \vdash^* (q, \epsilon, \epsilon)$, e se invece Y_i è un terminale, allora anche x_i è lo stesso terminale e usando la 2. abbiamo ancora $(q, x_i, Y_i) \vdash (q, \epsilon, \epsilon)$. Quindi si applica l'ipotesi induttiva e abbiamo che $A \Rightarrow Y_1 Y_2 \dots Y_k \xRightarrow{*} x_1 Y_2 \dots Y_k \xRightarrow{*} x_1 x_2 \dots Y_k \xRightarrow{*} \dots \xRightarrow{*} x_1 x_2 \dots x_k$

Se poniamo $A = S$ e $x = w$ abbiamo la tesi.

L'idea è semplice: l'automa simula una derivazione leftmost della grammatica. Tutte le parole derivabili possono essere derivate con una derivazione leftmost (eventualmente più di una), quindi non ci sono problemi. Accettare per pila vuota non è un problema. Il vice versa è più complesso.

Come funziona l'automa? Consuma eventualmente l'input e mette simboli sulla pila. Quando cambio stato? Quando elimino dalla pila un simbolo X ed eliminarlo significa anche eliminare tutto ciò che si è prodotto sulla pila dopo di lui e passo allo stato p da q , dopo la sostituzione di X con ϵ : $(q, xy, Y\alpha) \vdash^* (p, y, \alpha)$ (in mezzo la pila può crescere.)

Teorema 9 Sia $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$ un PDA che accetta per pila vuota. Allora esiste una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ tale che $\mathcal{L}(G) = \mathcal{N}_{PDA}(P)$.

Costruiamo la grammatica.

- $\mathbf{V} = \{[qXp] \mid p, q \in Q \text{ e } X \in \Gamma\} \cup \{S\}$
- supponiamo che $(p, Y_1Y_2 \cdots Y_k) \in \delta(q, a, X)$, allora, per ogni sequenza di stati $r_1, r_2, \dots, r_k$, abbiamo una produzione $[qXr_k] \rightarrow a[pY_1r_1][r_2Y_2r_3] \cdots [r_{k-1}Y_kr_k]$
- per ogni $p \in Q$, $S \rightarrow [q_0Z_0p]$

Mostriamo che $[qXp] \xRightarrow{*} w$ se e solo se $(q, w, X) \vdash^* (p, \epsilon, \epsilon)$.

Supponiamo $(q, w, X) \vdash^* (p, \epsilon, \epsilon)$.

Un passo: $(q, a, X) \vdash (p, \epsilon, \epsilon)$, quindi $(p, \epsilon) \in \delta(q, a, X)$ e abbiamo $[qXp] \rightarrow a$, con $a \in \Sigma$ oppure $a = \epsilon$, quindi $[qXp] \Rightarrow a$.

$(q, w, X) \vdash^* (p, \epsilon, \epsilon)$ sia di n passi. Il primo passo è $(q, w, X_0) \vdash (r_0, x, Y_1Y_2 \cdots Y_k)$ (poi si continua per ottenere (p, ϵ, ϵ)), $w = ax$ e $\delta(q, a, X)$ contiene $(r_0, Y_1Y_2 \cdots Y_k)$. Per costruzione $[qXr_k] \rightarrow a[r_0Y_1r_1][r_1Y_2r_2] \cdots [r_{k-1}Y_kr_k]$ sta nella grammatica e r_k nel nostro caso è p , mentre gli altri $r_1, \dots, r_{k-1}$ sono stati dell'automa.

I vari simboli Y_i sono stati poi eliminati dalla pila, quindi $(r_{i-1}, x_i, Y_i) \vdash^* (r_i, \epsilon, \epsilon)$ e per ipotesi induttiva abbiamo che $[r_{i-1}Y_i r_i] \xRightarrow{*} x_i$, e possiamo quindi ottenere:

$$\begin{aligned}
[qXr_k] &\Rightarrow a[r_0Y_1r_1][r_1Y_2r_2][r_2Y_3r_3] \cdots [r_{k-1}Y_kr_k] \\
&\xRightarrow{*} ax_1[r_1Y_2r_2][r_2Y_3r_3] \cdots [r_{k-1}Y_kr_k] \\
&\xRightarrow{*} ax_1x_2[r_2Y_3r_3] \cdots [r_{k-1}Y_kr_k] \\
&\vdots \\
&\xRightarrow{*} ax_1x_2 \cdots x_{k-1}[r_{k-1}Y_kr_k] \\
&\xRightarrow{*} ax_1x_2 \cdots x_{k-1}x_k = w
\end{aligned}$$

dove r_k è proprio p .

Supponiamo ora $[qXp] \xRightarrow{*} w$.

Un passo: $[qXp] \Rightarrow w$, e $[qXp] \rightarrow w$ è una produzione. Questa produzione esiste solo se $w = a$ oppure $w = \epsilon$ e abbiamo che $(p, \epsilon) \in \delta(q, a, X)$, quindi $(q, w, X) \vdash (p, \epsilon, \epsilon)$.

$[qXp] \xRightarrow{*} w$ sia una derivazione di n passi. Abbiamo, ponendo r_k uguale a p , che $[qXr_k] \Rightarrow a[r_0Y_1r_1][r_1Y_2r_2][r_2Y_3r_3] \cdots [r_{k-1}Y_kr_k] \xRightarrow{*} w$, e il primo passo deriva da $(r_0, Y_1Y_2 \cdots Y_k) \in \delta(q, a, X)$.

Scomponiamo w in $ax_1x_2\cdots x_k$ in modo che $[r_{i-1}Y_ir_i] \xRightarrow{*} x_i$. Per ipotesi induttiva sappiamo che $(r_{i-1}, x_i, Y_i) \vdash^* (r_i, \epsilon, \epsilon)$. Sapendo che da questo si può desumere, usando le proprietà di $\vdash^*$, $(r_{i-1}, x_i z, Y_i) \vdash^* (r_i, \epsilon z, \epsilon) = (r_i, z, \epsilon)$, possiamo ottenere

$$(q, ax_1x_2\cdots x_k, X) \vdash (r_0, x_1x_2\cdots x_k, Y_1Y_2\cdots Y_k) \vdash^* \cdots \vdash^* (r_{k-1}, x_k, Y_k) \vdash^* (p, \epsilon, \epsilon)$$

Consideriamo la grammatica $S \rightarrow [S] \mid [] \mid SS$, associamo l'automa.

$P = (\{q\}, \{[,]\}, \{S, [,]\}, \delta, q, S)$ dove

- $\delta(q, \epsilon, S) = \{(q, [S]), (q, []), (q, SS)\},$
- $\delta(q, [,]) = \{(q, \epsilon)\}$ e $\delta(q, [,]) = \{(q, \epsilon)\}$

Consideriamo la derivazione: $S \Rightarrow SS \Rightarrow [S]S \Rightarrow [[]]S \Rightarrow [[]] []$

Vediamo cosa succede in P :

$$\begin{aligned} (q, [[]] [], S) &\vdash (q, [[]] [], SS) \\ &\vdash (q, [[]] [], [S]S) \\ &\vdash (q, [[]] [], S[S]) \\ &\vdash (q, [[]] [], []S) \\ &\vdash (q, [] [], []S) \\ &\vdash (q, [] [],]S) \\ &\vdash (q, [] [],]S) \\ &\vdash (q, [], S) \\ &\vdash (q, [], []) \\ &\vdash (q, [],]) \\ &\vdash (q, \epsilon, \epsilon) \end{aligned}$$

Consideriamo l'automa $P = (\{q\}, \{i, e\}, \{Z\}, \delta, q, Z)$, dove δ è la seguente:

- $\delta(q, i, Z) = \{(q, ZZ)\},$
- $\delta(q, e, Z) = \{(q, \epsilon)\}$

Associamo la grammatica:

$G = (\{[qZq], S\}, \{i, e\}, \mathcal{P}, S)$ dove $\mathcal{P}$ contiene le produzioni:

- $[qZq] \rightarrow i[qZq][qZq], [qZq] \rightarrow e,$
- $S \rightarrow [qZq]$

Nell'automa: $(q, iee, Z) \vdash (q, ee, ZZ) \vdash (q, e, Z) \vdash (q, \epsilon, \epsilon)$

Nella grammatica: $S \Rightarrow [qZq] \Rightarrow i[qZq][qZq] \Rightarrow ie[qZq] \Rightarrow iee$

sono tutte le sequenze di *if else* illegali.

Determinismo Consideriamo il linguaggio $\{ww^R \mid w \in \{a, b\}^*\}$. Osserviamo che l'abbiamo già visto: infatti se applichiamo la reverse ad una parola del linguaggio abbiamo: $(ww^R)^R = (w^R)^R w^R$ e sapendo che $(w^R)^R = w$ (dimostratelo!) abbiamo che $(ww^R)^R = (w^R)^R w^R = ww^R$.

Chiaramente l'automa ad un certo punto deve decidere *sono in mezzo alla parola...* Non ho modo di evitare questa *guess*!

Un automa che accetta $\{ww^R \mid w \in \{a, b\}^*\}$ (per pila vuota):

$$(\{q_0, q\}, \{a, b\}, \{Z_0, A, B\}, \delta, q_0, Z_0)$$

dove

- $\delta(q_0, a, Z_0) = \{(q_0, A)\}$, $\delta(q_0, b, Z_0) = \{(q_0, B)\}$ e $\delta(q_0, \epsilon, Z_0) = \{(q_0, \epsilon)\}$
- $\delta(q_0, a, A) = \{(q_0, AA), (q, \epsilon)\}$, $\delta(q_0, b, A) = \{(q_0, BA), (q, \epsilon)\}$,
 $\delta(q_0, b, B) = \{(q_0, BB), (q, \epsilon)\}$ e $\delta(q_0, a, B) = \{(q_0, AB), (q, \epsilon)\}$
- $\delta(q, a, A) = \{(q, \epsilon)\}$ e $\delta(q, b, B) = \{(q, \epsilon)\}$

Consideriamo il linguaggio $\{wcw^R \mid w \in \{a, b\}^*\}$. Qui l'automa non deve decidere *sono in mezzo alla parola*. Un automa che accetta $\{wcw^R \mid w \in \{a, b\}^*\}$ (per pila vuota):

$$(\{q_0, q\}, \{a, b, c\}, \{Z_0, A, B\}, \delta, q_0, Z_0)$$

dove

- $\delta(q_0, a, Z_0) = \{(q_0, A)\}$, $\delta(q_0, b, Z_0) = \{(q_0, B)\}$ e $\delta(q_0, \epsilon, Z_0) = \{(q_0, \epsilon)\}$
- $\delta(q_0, a, A) = \{(q_0, AA)\}$, $\delta(q_0, b, A) = \{(q_0, BA)\}$,
 $\delta(q_0, b, B) = \{(q_0, BB)\}$ e $\delta(q_0, a, B) = \{(q_0, AB)\}$
- $\delta(q_0, c, A) = \{(q, A)\}$ e $\delta(q_0, c, B) = \{(q, B)\}$
- $\delta(q, a, A) = \{(q, \epsilon)\}$ e $\delta(q, b, B) = \{(q, \epsilon)\}$

Questo automa ha la caratteristica che la δ si comporta come una funzione che soddisfa alcuni vincoli:

- $|\delta(q, a, X)| \leq 1$, per ogni $a \in \Sigma \cup \{\epsilon\}$, $X \in \Gamma$, $q \in Q$
- se $\delta(q, a, X) \neq \emptyset$, per $a \in \Sigma$, $X \in \Gamma$, $q \in Q$, allora $\delta(q, \epsilon, X)$ è vuoto

Esercizi

1. definite un PDA per $\{a^i b^j c^k \mid i = j \text{ oppure } j = k\}$
2. definite un PDA per $\{x \in \{a, b\}^* \mid \text{non esiste } w \in \{a, b\}^* \text{ tale che } ww = x\}$
3. supponete che $\mathcal{L}$ non contenga ϵ e che venga accettato da P per stack vuoto. Modificare P in modo che accetti anche ϵ per stack vuoto.
4. se nel primo esercizio il PDA accetta per stack vuoto, convertitelo in uno che accetta per stato finale

5. mostrate che se P è un automa a pila, allora esiste un PDA P' con due soli simboli di stack che accetta lo stesso linguaggio: $\mathcal{L}_{PDA}(P) = \mathcal{L}_{PDA}(P')$
6. considerate $aaaabbbbcc \in \{a^i b^j c^k \mid i = j \text{ oppure } j = k\}$. Descrivete la computazione di accettazione dell'automa che avete costruito
7. Convertire la grammatica $S \rightarrow 0S1 \mid A, A \rightarrow 1A0 \mid S \mid \epsilon$ in un PDA che accetta lo stesso linguaggio
8. Mostrate che se P è un PDA, allora esiste un PDA P' con un solo stato tale che $\mathcal{N}_{PDA}(P) = \mathcal{N}_{PDA}(P')$
9. Costruire un PDA che riconosce $\{0^n 1^m \mid n \leq m \leq 2n\}$ e poi associate la grammatica che lo genera
10. Costruire un PDA che riconosce $\{a^i b^j c^k \mid i = 2j \text{ oppure } j = 2k\}$ e poi associate la grammatica che lo genera
11. Convertite in PDA le grammatiche che avete trovato negli esercizi precedenti
12. Convertite in grammatiche gli automi costruiti a lezione

Forme normali

Se un linguaggio è context free, allora ha una grammatica, e in particolare ha una grammatica le cui produzioni hanno tutte una forma particolare. Quelle che vediamo sono due, la forma normale di *Chomsky* e quella di *Greibach*. La prima forma normale è tale che c'è una precisa relazione tra la lunghezza della derivazione e l'albero sintattico della parola derivata, mentre la forma normale di Greibach è usata soprattutto dagli algoritmi di parsing.

0.2 Forma normale di Chomsky

Consideriamo solo linguaggi context free che non contengono la parola vuota. Questa non è una limitazione, come subito vediamo.

Teorema 10 *Sia $\mathcal{L}$ un linguaggio context free tale che $\epsilon \notin \mathcal{L}$. Allora esiste una grammatica G che genera il linguaggio $\mathcal{L} \cup \{\epsilon\}$*

Dato che $\mathcal{L}$ è context free è generato da una grammatica G' . Quindi $\mathcal{L}(G') = \mathcal{L}$. Sia $G' = (\mathbf{V}, \Sigma, \mathcal{P}, S)$, definiamo $G = (\mathbf{V}, \Sigma, \mathcal{P} \cup \{S \rightarrow \epsilon\}, S)$. Ovviamente $\mathcal{L} \subseteq \mathcal{L}(G)$ e inoltre $S \Rightarrow \epsilon$ usando la nuova produzione.

Definizione 22 *Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. G è in forma normale di Chomsky (CNF) se e solo se tutte le produzioni sono della forma $A \rightarrow BC$ o $A \rightarrow a$, dove $A, B, C \in \mathbf{V}$ e $a \in \Sigma$.*

Il teorema che vogliamo provare dice che ogni grammatica che genera un linguaggio context free senza ‘ ϵ ’ può essere messa in CFN. Per far questo dobbiamo:

- Eliminare i simboli inutili, cioè quelli che non appaiono in nessuna derivazione $S \xRightarrow{*} w$, per S simbolo iniziale e $w \in \Sigma^*$
- Eliminare le produzioni della forma $A \rightarrow \epsilon$
- Eliminare le produzioni unità (i.e., della forma $A \rightarrow B$, dove $A, B \in \mathbf{V}$)
- Eliminare le produzioni con più di due non terminali sulla destra

Eliminazione dei simboli inutili

Definizione 23 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. Un simbolo $V \in \mathbf{V}$ è inutile se e solo se $\forall w \in \mathcal{L}(G)$ e per ogni derivazione $S \xRightarrow{*} w$, V non compare nella derivazione.

Possiamo eliminare i simboli inutili:

Teorema 11 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica, e sia $\mathcal{L} = \mathcal{L}(G) \neq \emptyset$. Allora esiste una grammatica G' senza simboli inutili tale che $\mathcal{L} = \mathcal{L}(G')$

Dato che $\mathcal{L}(G) \neq \emptyset$ abbiamo almeno una derivazione. Se G non ha simboli inutili allora siamo a posto. Altrimenti sia $\mathbf{V}_{in} \subseteq \mathbf{V}$ l'insieme dei simboli inutili. Mostriamo che $G' = (\mathbf{V} \setminus \mathbf{V}_{in}, \Sigma, \mathcal{P}', S)$, dove $\mathcal{P}' = \{A \rightarrow \alpha \in \mathcal{P} \mid A \notin \mathbf{V}_{in} \wedge \alpha \in (\Sigma \cup \mathbf{V} \setminus \mathbf{V}_{in})^*\}$, è tale che $\mathcal{L} = \mathcal{L}(G')$. Chiaramente $\mathcal{L}(G') \subseteq \mathcal{L}$. Mostriamo che $\mathcal{L} \subseteq \mathcal{L}(G')$. Prendiamo $w \in \mathcal{L}(G)$, quindi $S \xRightarrow{*} w$. Questa derivazione è anche una derivazione di G' , dato che non compaiono i simboli inutili che abbiamo levato. Quindi $w \in \mathcal{L}(G')$.

Come trovare i simboli inutili Partiamo dalla grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$. Cerchiamo i simboli utili:

- $X \in \mathbf{V}$ è un *generatore* se esiste una parola $w \in \Sigma^*$ tale che $X \xRightarrow{*} w$
- X è raggiungibile se esiste una derivazione $S \xRightarrow{*} \alpha X \beta$

I simboli utili sono tutti quelli che sono generatori e raggiungibili. Per trovarli procediamo a ritroso: tutte le produzioni che riscrivono in stringhe di soli terminali sono generatori. Se $A \rightarrow \alpha$ è una produzione e tutti i simboli di α sono terminali o generatori, allora A è un generatore. Infine basta osservare che un simbolo X (generatore) è raggiungibile se e solo se S è un generatore.

Consideriamo

$$G = (\{A, B, S\}, \{0, 1, 2\}, \{S \rightarrow 0S1 \mid A, A \rightarrow 2, B \rightarrow 1\}, S)$$

Si vede che il linguaggio è $\{0^n 21^n \mid n \geq 0\}$.

Chiaramente B è inutile: infatti non è raggiungibile, anche se è un generatore. Se le produzioni fossero $\{S \rightarrow 0S1 \mid A, A \rightarrow 2, B \rightarrow B\}$ allora abbiamo che B non è neanche generatore.

La grammatica equivalente è

$$G' = (\{A, S\}, \{0, 1, 2\}, \{S \rightarrow 0S1 \mid A, A \rightarrow 2\}, S)$$

.

Eliminare le produzioni della forma $A \rightarrow \epsilon$ Cerchiamo tutti i simboli di variabile di una grammatica che sono *annullabili*. Diamone la definizione.

Definizione 24 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. Un simbolo $A \in \mathbf{V}$ è annullabile se

- esiste una produzione $A \rightarrow \epsilon$, o
- esiste una produzione $A \rightarrow Y_1 Y_2 \dots Y_k$ e ogni Y_i è annullabile

Non deve sfuggire che questa definizione ci dà un modo per calcolare quali sono i simboli annullabili. Ricordiamoci che stiamo considerando linguaggi senza la parola vuota.

Teorema 12 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica senza simboli inutili, e sia $\mathcal{L} = \mathcal{L}(G)$. Allora esiste una grammatica G' dove non ci sono produzioni $A \rightarrow \epsilon$ tale che $\mathcal{L} = \mathcal{L}(G')$.

Se G non ha simboli annullabili allora il teorema è dimostrato. Supponiamo che il simbolo A sia annullabile (la generalizzazione a più simboli annullabili è banale). Consideriamo tutte le produzioni $V \rightarrow \alpha$ tali che A compare in α , i.e., $\alpha = \beta A \gamma$. Aggiungiamo le produzioni $V \rightarrow \beta \gamma$ se $V \rightarrow \beta A \gamma \in \mathcal{P}$. Mostriamo che $\mathcal{L}(G) = \mathcal{L}(G')$. Prendiamo una derivazione di w che usa $A \rightarrow \epsilon$. Chiaramente abbiamo $S \xRightarrow{*}_G \alpha_1 V \alpha_2 \Rightarrow_G \alpha_1 \beta A \gamma \alpha_2 \Rightarrow_G \alpha_1 \beta \gamma \alpha_2 \xRightarrow{*}_G w$ per una qualche produzione $V \rightarrow \beta A \gamma$. Ma la derivazione $S \xRightarrow{*}_{G'} \alpha_1 V \alpha_2 \Rightarrow_{G'} \alpha_1 \beta \gamma \alpha_2 \xRightarrow{*}_{G'} w$ è una derivazione di G' .

Per il vice versa basta osservare che se usiamo una produzione $V \rightarrow \beta \gamma$ che non compare in $\mathcal{P}$, allora, per definizione di G' , abbiamo che G contiene le produzioni $V \rightarrow \beta A \gamma$ e $A \rightarrow \epsilon$, e quindi alla derivazione $V \Rightarrow_{G'} \beta \gamma$ corrisponde $V \Rightarrow_G \beta A \gamma \Rightarrow_G \beta \gamma$.

Osserviamo che G' potrebbe avere simboli inutili (le variabili delle produzioni $A \rightarrow \epsilon$, che sappiamo eliminare).

Consideriamo

$$G = (\{S, A\}, \{0, 1, \}, \{S \rightarrow 0A1, A \rightarrow 0A1 \mid \epsilon\}, S)$$

Si vede che il linguaggio è $\{0^n 1^n \mid n \geq 1\}$ (quindi non contiene ϵ). Chiaramente A è annullabile (grazie a $A \rightarrow \epsilon$). Aggiungiamo le produzioni $A \rightarrow 01$ (ottenuta da $A \rightarrow \epsilon$ e $A \rightarrow 0A1$) e $S \rightarrow 01$ (ottenuta da $A \rightarrow \epsilon$ e $S \rightarrow 0A1$) e leviamo solo quella $A \rightarrow \epsilon$. La grammatica equivalente è

$$G' = (\{A, S\}, \{0, 1\}, \{S \rightarrow 0A1 \mid 01, A \rightarrow 0A1 \mid 01\}, S)$$

.

Eliminare le produzioni della forma $A \rightarrow B$, dove $A, B \in \mathbf{V}$ Iniziamo con la definizione di coppia unitaria.

Definizione 25 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica. Una coppia unitaria è un elemento $(X, Y) \in \mathbf{V} \times \mathbf{V}$ tale che $X \xRightarrow{*} Y$.

Una coppia unitaria è una coppia di non terminali che sono derivabili uno dall'altro. Come calcoliamo le coppie unitarie?

- (A, A) è una coppia unitaria
- se (A, B) è una coppia unitaria e $B \rightarrow C$ con $B, C \in \mathbf{V}$ è una produzione, allora anche (A, C) è una coppia unitaria

Per ogni coppia unitaria (A, B) si aggiunge la produzione $A \rightarrow \alpha$, dove $B \rightarrow \alpha$ è una produzione non unitaria in $\mathcal{P}$.

Teorema 13 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica con soli simboli utili, e sia $\mathcal{L} = \mathcal{L}(G)$. Allora esiste una grammatica G' dove non ci sono produzioni unitarie tale che $\mathcal{L} = \mathcal{L}(G')$.

Se G non ha coppie unitarie tranne che (A, A) allora non abbiamo da fare nulla (se ci fosse la produzione $A \rightarrow A$ possiamo eliminarla senza problemi). Supponiamo che ci sia la coppia (A, B) , con $A \neq B$. Otteniamo la G' aggiungendo la produzioni $A \rightarrow \alpha$, dove $B \rightarrow \alpha$ è una produzione non unitaria in $\mathcal{P}$.

Mostriamo che $\mathcal{L}(G) = \mathcal{L}(G')$. Prendiamo una derivazione di w dove si usa la produzione $A \rightarrow B$. Chiaramente abbiamo $S \xRightarrow{*}_G \alpha_1 A \alpha_2 \Rightarrow_G \alpha_1 B \alpha_2 \Rightarrow_G \alpha_1 \alpha \alpha_2 \xRightarrow{*}_G w$ per una qualche produzione $B \rightarrow \alpha$. Ma la derivazione $S \xRightarrow{*}_{G'} \alpha_1 A \alpha_2 \Rightarrow_{G'} \alpha_1 \alpha \alpha_2 \xRightarrow{*}_{G'} w$ è una derivazione di G' .

Per il vice versa basta osservare che se usiamo una produzione $B \rightarrow \alpha$ che non compare in $\mathcal{P}$, allora, per definizione di G' , abbiamo che G contiene le produzioni $A \rightarrow B$ e $B \rightarrow \alpha$, e quindi alla derivazione $A \Rightarrow_{G'} \alpha$ corrisponde $A \Rightarrow_G B \Rightarrow_G \alpha$.

Osserviamo che G' potrebbe avere simboli inutili (le variabili delle produzioni $B \rightarrow \alpha$, che sappiamo eliminare).

Consideriamo la grammatica

$$G = (\{E\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid (E) \mid E^* \mid E + E, \mid E \cdot E\}, E)$$

e la sua versione non ambigua:

$$G = (\{E, T, F, K\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow T \mid E + T, \\ T \rightarrow F \mid T \cdot F, F \rightarrow K \mid F^*, K \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon\}, E)$$

Attenzione, non ci sono simboli inutili o produzioni $A \rightarrow \epsilon$: qui l' ϵ è un terminale!

Le coppie unitarie sono $(E, T), (E, F), (E, K), (T, F), (T, K), (F, K)$, infatti

- da (E, T) otteniamo la produzione $E \rightarrow T \cdot F$, da (E, F) otteniamo $E \rightarrow F^*$,
- da (E, K) otteniamo $E \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon$
- da (T, F) otteniamo $T \rightarrow F^*$
- da (T, K) otteniamo $T \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon$
- da (F, K) otteniamo $F \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon$

queste nuove produzioni sostituiscono quelle unitarie, quindi:

$$G = (\{E, T, F, K\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow T \cdot F \mid E + T \mid F^* \mid (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, \\ T \rightarrow F^* \mid T \cdot F \mid (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, F \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid F^*, K \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon\}, E)$$

Produzioni con soli non terminali o soli terminali Prendiamo una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ e una produzione $Y \rightarrow X_1 X_2 \cdots X_n$. Se X_i è una variabile, la lasciamo. Se invece X_i è un simbolo terminale, introduciamo una nuova variabile $\bar{X}_i$, che sostituiamo a X_i nel lato destro della produzione, e introduciamo una nuova produzione $\bar{X}_i \rightarrow X_i$

Chiaramente la nuova grammatica G' così ottenuta è tale che $\mathcal{L}(G) = \mathcal{L}(G')$. Vediamo quindi come eliminare le produzioni con più di due non terminali sulla destra: l'idea è ovvia: rimpiazzo la produzione con una serie di produzioni che simulano l'effetto della produzione che devo rimpiazzare, *introducendo* nuove variabili, la cui dimostrazione è ovvia e quindi qui la omettiamo.

Teorema 14 Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica e $A \rightarrow \alpha$ una produzione in $\mathcal{P}$ tale che $\alpha \in \mathbf{V}^*$ e $|\alpha| > 2$. Sia $\alpha = X\beta$, Allora possiamo introdurre una nuova variabile $[\beta]$ e la produzione $[\beta] \rightarrow \beta$ e sostituire $A \rightarrow \alpha$ con $A \rightarrow X[\beta]$ tali che $A \Rightarrow_G \alpha$ se e solo se $A \Rightarrow_{G'} X[\beta] \Rightarrow_{G'} X\beta$.

Partiamo da

$$G = (\{E, T, F, K\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow T \cdot F \mid E + T \mid F^* \mid (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, \\ T \rightarrow F^* \mid T \cdot F \mid (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, F \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid F^*, K \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon\}, E)$$

osserviamo che K è inutile (non è raggiungibile)... quindi otteniamo

$$G' = (\{E, T, F\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{E \rightarrow T \cdot F \mid E + T \mid F^* \mid (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, \\ T \rightarrow F^* \mid T \cdot F \mid (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, F \rightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid F^*\}, E)$$

Introduciamo produzioni per ogni terminale (e sostituiamo i terminali nelle produzioni):
 $A \rightarrow \mathbf{a}, B \rightarrow \mathbf{b}, C \rightarrow +, D \rightarrow \cdot, G \rightarrow *, H \rightarrow (, I \rightarrow)$ e $L \rightarrow \epsilon$. Otteniamo

$$G'' = (\{E, T, F, A, B, C, D, G, H, I, L\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{A \rightarrow \mathbf{a}, B \rightarrow \mathbf{b}, C \rightarrow +, \\ D \rightarrow \cdot, G \rightarrow *, H \rightarrow (, I \rightarrow), L \rightarrow \epsilon, E \rightarrow TDF \mid ECT \mid FG \mid HEI \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, \\ T \rightarrow FG \mid TDF \mid HEI \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, F \rightarrow HEI \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid FG\}, E)$$

Abbiamo

$$G'' = (\{E, T, F, A, B, C, D, G, H, I, L\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \{A \rightarrow \mathbf{a}, B \rightarrow \mathbf{b}, C \rightarrow +, \\ D \rightarrow \cdot, G \rightarrow *, H \rightarrow (, I \rightarrow), L \rightarrow \epsilon, E \rightarrow TDF \mid ECT \mid FG \mid HEI \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, \\ T \rightarrow FG \mid TDF \mid HEI \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, F \rightarrow HEI \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid FG\}, E)$$

Dobbiamo sostituire le produzioni che violano la proprietà e che sono le seguenti: $E \rightarrow TDF \mid ECT \mid HEI$, $T \rightarrow TDF \mid HEI$ e $F \rightarrow HEI$. Introduciamo i non terminali $[DF]$, $[CT]$, $[EI]$ e le produzioni $[DF] \rightarrow DF$, $[CT] \rightarrow CT$, $[EI] \rightarrow EI$ e per produzioni di con più simboli di variabile abbiamo $E \rightarrow T[DF] \mid E[CT] \mid H[EI]$, $T \rightarrow T[DF] \mid H[EI]$ e $F \rightarrow H[EI]$

Abbiamo

$$\mathbf{G} = (\{E, T, F, A, B, C, D, G, H, I, L, [DF], [CT], [EI]\}, \{\epsilon, \emptyset, \mathbf{a}, \mathbf{b}, +, *, \cdot, (,)\}, \\ \{[DF] \rightarrow DF, [CT] \rightarrow CT, [EI] \rightarrow EI, A \rightarrow \mathbf{a}, B \rightarrow \mathbf{b}, C \rightarrow +, \\ D \rightarrow \cdot, G \rightarrow *, H \rightarrow (, I \rightarrow), L \rightarrow \epsilon, E \rightarrow T[DF] \mid E[CT] \mid FG \mid H[EI] \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, \\ T \rightarrow FG \mid T[DF] \mid H[EI] \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon, F \rightarrow H[EI] \mid \mathbf{a} \mid \mathbf{b} \mid \emptyset \mid \epsilon \mid FG\}, E)$$

che è in forma normale di Chomsky. Se $\mathcal{L}$ contenesse ϵ , allora troviamo una grammatica in forma normale di Chomsky per $\mathcal{L} \setminus \{\epsilon\}$ e aggiungiamo la produzione $S \rightarrow \epsilon$. Ovviamente la nuova grammatica genera anche ϵ .

Forma normale di Greibach Esiste anche un'altra forma normale. In questo caso le produzioni sono del tipo $A \rightarrow a\alpha$ dove $a \in \Sigma$ e $\alpha \in \mathbf{V}^*$.

Osserviamo prima alcuni fatti:

Lemma 1 Consideriamo una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ in cui tutti i simboli sono utili. Consideriamo una produzione $A \rightarrow \alpha_1 B \alpha_2$ e le produzioni $B \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$. Allora possiamo sostituire $A \rightarrow \alpha_1 B \alpha_2$ con $A \rightarrow \alpha_1 \beta_1 \alpha_2 \mid \alpha_1 \beta_2 \alpha_2 \mid \dots \mid \alpha_1 \beta_n \alpha_2$ e la nuova grammatica $G' = (\mathbf{V}, \Sigma, \mathcal{P}', S)$ così ottenuta genera lo stesso linguaggio.

La dimostrazione è ovvia.

$\mathcal{L}(G') \subseteq \mathcal{L}(G)$ dato che ad ogni derivazione in G' che usa $A \rightarrow \alpha_1\beta_i\alpha_2$ corrisponde la derivazione in G che usa $A \rightarrow \alpha_1B\alpha_2$ e $B \rightarrow \beta_i$ cioè $A \Rightarrow \alpha_1B\alpha_2 \Rightarrow \alpha_1\beta_i\alpha_2$.

Per il vice versa basta osservare che se si usa $A \rightarrow \alpha_1B\alpha_2$ in una derivazione di G , prima o poi bisognerà usare anche $B \rightarrow \beta_i$, quindi in G' usiamo direttamente $A \rightarrow \alpha_1\beta_i\alpha_2$

Lemma 2 Consideriamo una grammatica $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ in cui tutti i simboli sono utili. Consideriamo tutte le produzioni $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_k$ e siano tutte le altre produzioni $A \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$ che riscrivono A . Definiamo $G' = (\mathbf{V} \cup \{B\}, \Sigma, \mathcal{P}', S)$, con $B \notin \mathbf{V}$ e $\mathcal{P}'$ ottenuto rimpiazzando tutte le produzioni che riscrivono A con le seguenti: $A \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$ e $A \rightarrow \beta_1B \mid \beta_2B \mid \dots \mid \beta_nB$ e per la nuova variabile B si introducono le produzioni $B \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$ e $B \rightarrow \alpha_1B \mid \alpha_2B \mid \dots \mid \alpha_kB$. Allora $\mathcal{L}(G') = \mathcal{L}(G)$

Una derivazione leftmost in G che usa A è:

$$A \Rightarrow A\alpha_{i_1} \Rightarrow A\alpha_{i_2}\alpha_{i_1} \Rightarrow \dots \Rightarrow A\alpha_{i_{p-2}}\dots\alpha_{i_2}\alpha_{i_1} \Rightarrow A\alpha_{i_{p-1}}\alpha_{i_{p-2}}\dots\alpha_{i_2}\alpha_{i_1} \Rightarrow \dots \Rightarrow \beta_{i_p}\alpha_{i_{p-1}}\alpha_{i_{p-2}}\dots\alpha_{i_2}\alpha_{i_1}$$

e a questa associamo in G' la derivazione

$$A \Rightarrow \beta_{i_p}B \Rightarrow \beta_{i_p}\alpha_{i_{p-1}}B \Rightarrow \beta_{i_p}\alpha_{i_{p-1}}\alpha_{i_{p-2}}B \Rightarrow \dots \Rightarrow \beta_{i_p}\alpha_{i_{p-1}}\alpha_{i_{p-2}}\dots\alpha_{i_2}B \Rightarrow \dots \Rightarrow \beta_{i_p}\alpha_{i_{p-1}}\alpha_{i_{p-2}}\dots\alpha_{i_2}\alpha_{i_1}$$

Anche il viceversa vale, e viene dimostrato nello stesso modo, quindi possiamo concludere che $\mathcal{L}(G') = \mathcal{L}(G)$.

Teorema 15 Ogni linguaggio $\mathcal{L}$ context free senza la parola vuota può essere generato da una grammatica in cui le produzioni sono della forma $A \rightarrow a\alpha$, dove a è un terminale e α è una stringa, eventualmente vuota, di variabili.

Partiamo da $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ in forma normale di Chomsky e numeriamo le variabili: $\mathbf{V} = \{A_1, A_2, \dots, A_m\}$. Per prima cosa facciamo in modo che $A_i \rightarrow A_j\gamma$ sia tale che $j > i$.

Supponiamo di averlo fatto fino a $1 \leq k-1 < m$ e facciamolo per k . Se $A_k \rightarrow A_j\gamma$ e $j < k$ allora rimpiazziamo la produzione sostituendo ogni A_j con il lato destro delle produzioni di A_j che sono tali che $A_j \rightarrow A_l\gamma_l$ e $l > j$ (Lemma 2). Se ripetiamo questo $k-1$ volte al massimo, arriviamo a produzioni della forma $A_k \rightarrow A_n\gamma$ con $n \geq k$. Le produzioni in cui $k = n$ vengono rimpiazzate come visto nel lemma 1.

Alla fine del procedimento abbiamo che le produzioni sono della forma: $A_i \rightarrow A_j\gamma$, con $i < j$, $A_i \rightarrow a\gamma$, con $a \in \Sigma$ e $B_i \rightarrow \gamma$ con $\gamma \in \mathbf{V} \cup \{B_1, B_2, \dots, B_{i-1}\}^*$ (le nuove variabili sono quelle introdotte secondo il Lemma 2). Siamo partiti da una grammatica in forma normale di Chomsky, quindi le produzioni

sono del tipo $A_i \rightarrow a$ o $A_i \rightarrow A_j A_l$. Ora nessuna delle produzioni che iniziano per B_i può iniziare con B_j , quindi tutte queste produzioni iniziano con un terminale o A_j , ed una nuova applicazione del lemma 2 termina la costruzione.

0.3 Operazioni sui linguaggi context free

Vediamo ora le proprietà di chiusura. Osserviamo che alcune delle proprietà di chiusura dei linguaggi regolari valgono anche per quelli context free: unione, concatenazione, star di Kleene, intersezione con un linguaggio regolare, omomorfismo e omomorfismo inverso, vedremo anche perché.

Sostituzione Una *sostituzione* è una funzione che associa ad ogni elemento di un alfabeto un linguaggio (possibilmente su un altro alfabeto).

Definizione 26 Dato un alfabeto Σ e dato un insieme di linguaggi (non necessariamente tutti sullo stesso alfabeto), una sostituzione è una funzione $s : \Sigma \rightarrow \{\mathcal{L} \mid \mathcal{L} \text{ è un linguaggio}\}$. Con $s(a)$ indichiamo il linguaggio $\mathcal{L}_a$ associato ad a . Se consideriamo una parola $w \in \Sigma^*$, $s(w)$ è il linguaggio ottenuto nel seguente modo: $w = a_1 a_2 \dots a_n$, allora $s(w) = s(a_1) s(a_2) \dots s(a_n)$, cioè la concatenazione di tutti i linguaggi associati ad ogni terminale. Dato $\mathcal{L} \subseteq \Sigma^*$, $s(\mathcal{L}) = \bigcup_{w \in \mathcal{L}} s(w)$.

Mostriamo che i linguaggi context-free sono chiusi per sostituzione.

Teorema 16 Sia $\mathcal{L}$ un linguaggio context free su Σ , e sia s una sostituzione tale che $s(a)$ è un linguaggio context free per ogni $a \in \Sigma$. Allora $s(\mathcal{L})$ è un linguaggio context free.

Dato che $\mathcal{L}$ è context free, esiste una grammatica $G = (\mathcal{V}, \Sigma, \mathcal{P}, S)$ tale che $\mathcal{L}(G) = \mathcal{L}$. Anche $s(a)$ è un linguaggio context free (per ogni $a \in \Sigma$) e quindi $s(a) = \mathcal{L}(G_a)$ dove $G_a = (\mathcal{V}_a, \Sigma_a, \mathcal{P}_a, S_a)$.

Definiamo la seguente grammatica: $G' = (\mathcal{V} \cup (\bigcup_{a \in \Sigma} \mathcal{V}_a), \bigcup_{a \in \Sigma} \Sigma_a, \mathcal{P}' \cup (\bigcup_{a \in \Sigma} \mathcal{P}_a), S)$ dove $\mathcal{P}'$ è ottenuto da $\mathcal{P}$ sostituendo ogni occorrenza di un terminale $a \in \Sigma$ con S_a .

Mostriamo che $w \in \mathcal{L}(G')$ se e solo se $w \in s(\mathcal{L})$.

(SE) Prendiamo $w \in s(\mathcal{L})$. Allora esiste una parola $a_1 a_2 \dots a_n \in \mathcal{L}$ tale che w è la concatenazione di parole $x_i \in s(a_i)$. Possiamo derivare w in questo modo: $S \xrightarrow{*}_{G'} S_{a_1} S_{a_2} \dots S_{a_n}$ (che corrisponde a $S \xrightarrow{*}_G a_1 a_2 \dots a_n$) e da ogni $S_{a_i} \xrightarrow{*}_{G_{a_i}} x_i$. Ora la concatenazione delle x_i mi dà proprio w .

(SOLO SE) Prendiamo $w \in \mathcal{L}(G')$. La possiamo derivare applicando prima tutte le produzioni in $\mathcal{P}'$ ottenendo quindi $S \xrightarrow{*}_{G'} S_{a_1} S_{a_2} \dots S_{a_n}$. Questa parte di derivazione ha una corrispondenza alla derivazione in

G della parola $x = a_1 a_2 \cdots a_n$. Dato che ogni S_{a_i} genera una parola in $s(a_i)$, w è la concatenazione di parole nei linguaggi associati ad ogni terminale, quindi $w \in s(x)$ e quindi anche $w \in s(\mathcal{L})$.

Vediamo come applicare questo risultato

Teorema 17 *I linguaggi context free sono chiusi rispetto all'unione, concatenazione, star di Kleene, chiusura positiva e omomorfismo.*

- unione: siano $\mathcal{L}_1$ e $\mathcal{L}_2$ due linguaggi context free. $\mathcal{L}_1 \cup \mathcal{L}_2$ è ottenuto dalla sostituzione sul linguaggio $\{1, 2\}$ definita come $s(1) = \mathcal{L}_1$ e $s(2) = \mathcal{L}_2$.
- concatenazione: siano $\mathcal{L}_1$ e $\mathcal{L}_2$ due linguaggi context free. $\mathcal{L}_1 \cdot \mathcal{L}_2$ è ottenuto dalla sostituzione sul linguaggio $\{12\}$ definita come $s(1) = \mathcal{L}_1$ e $s(2) = \mathcal{L}_2$.
- star di Kleene: sia $\mathcal{L}$ un linguaggio context free. $\mathcal{L}^*$ è ottenuto dalla sostituzione sul linguaggio $\{1\}^*$ definita come $s(1) = \mathcal{L}$.
- chiusura positiva: : sia $\mathcal{L}$ un linguaggio context free. $\mathcal{L}^+$ è ottenuto dalla sostituzione sul linguaggio $\{1\}^+$ definita come $s(1) = \mathcal{L}$.
- omomorfismo: ovvio, dato che è una sostituzione che associa ad ogni simbolo una sola parola.

Inversione

Teorema 18 *I linguaggi context free sono chiusi rispetto all'inversione.*

Sia $\mathcal{L}$ un linguaggio context free. Mostriamo che anche $\mathcal{L}^R$ è context free.

Dato che $\mathcal{L}$ è context free, esiste una grammatica $G = (\mathcal{V}, \Sigma, \mathcal{P}, S)$ tale che $\mathcal{L} = \mathcal{L}(G)$. Costruiamo la grammatica $G^R = (\mathcal{V}, \Sigma, \mathcal{P}^R, S)$ dove $\mathcal{P}^R = \{A \rightarrow \alpha^R \mid A \rightarrow \alpha \in \mathcal{P}\}$.

E' facile vedere che $\mathcal{L}^R = \mathcal{L}(G^R)$.

Intersezione con un linguaggio regolare

Teorema 19 *I linguaggi context free sono chiusi rispetto all'intersezione con i linguaggi regolari.*

Consideriamo un linguaggio context free $\mathcal{L}$ ed un linguaggio regolare $\mathcal{R}$. Abbiamo per $\mathcal{L}$ un PDA $P = (Q_L, \Sigma, \Gamma, \delta_L, q_{0_L}, Z_0, F_L)$ che accetta $\mathcal{L}$ per stato finale e per $\mathcal{R}$ un DFA $A = (Q_R, \Sigma, \delta_R, q_{0_R}, F_R)$

Costruiamo un PDA per $\mathcal{L} \cap \mathcal{R}$ come segue:

$$P' = (Q_L \times Q_R, \Sigma, \Gamma, \delta, (q_{0_L}, q_{0_R}), Z_0, F_L \times F_R)$$

dove $\delta'((q, q'), a, X)$ è definita come segue:

- $\delta((q, q'), a, X) = \{((p, p'), \alpha) \mid (p, \alpha) \in \delta_L(q, a, X) \text{ e } p' = \hat{\delta}_R(q', a)\}$ per ogni $a \in \Sigma \cup \{\epsilon\}$, $X \in \Gamma$ e $(q, q') \in Q_L \times Q_R$

L'idea è che faccio andare i due automi accoppiati, osservando che il DFA deve potersi fermare nel caso l'automa a pila non consumi l'input.

Si vede facilmente che $((q_{0_L}, q_{0_R}), w, Z_0) \vdash^* ((p, p'), \epsilon, \gamma)$, con $(p, p') \in F_L \times F_R$ se e solo se $(q_{0_L}, w, Z_0) \vdash^* (p, \epsilon, \gamma)$ e $\hat{\delta}_R(q_{0_R}, w) = p'$. Quindi $\mathcal{L}_{PDA}(P') = \mathcal{L} \cap \mathcal{R}$.

Omomorfismo inverso

Teorema 20 *I linguaggi context free sono chiusi rispetto all'omomorfismo inverso.*

Consideriamo un linguaggio context free $\mathcal{L}$ e sia h un omomorfismo. Mostriamo che $h^{-1}(\mathcal{L})$ è context free. h associa simboli dell'alfabeto Σ a parole sull'alfabeto Δ , quindi $\mathcal{L} \subseteq \Delta^*$. Abbiamo per $\mathcal{L}$ un PDA $P = (Q, \Delta, \Gamma, \delta, q_0, Z_0, F)$ che accetta $\mathcal{L}$ per stato finale.

Costruiamo un PDA per $h^{-1}(\mathcal{L})$ come segue: $P' = (Q', \Sigma, \Gamma, \delta', (q_0, \epsilon), Z_0, F \times \{\epsilon\})$ e

- * $Q' = Q \times \{x \in \Delta^* \mid x \text{ è un suffisso di } h(a), \text{ con } a \in \Sigma\}$
- * $\delta'((q, x), a, X)$ è definita come segue:
 - $\delta'((q, \epsilon), a, X) = \{((q, h(a)), X)\}$ per ogni $a \in \Sigma$, $X \in \Gamma$ e $q \in Q$
 - se $(q', \alpha) \in \delta(q, b, X)$, con $b \in \Delta$, $X \in \Gamma$ e $q \in Q$ allora $\delta'((q, bx), \epsilon, X)$ contiene la coppia $((q', x), \alpha)$.

L'idea è che gli stati contengono un buffer in cui memorizzo parti della parola che devo riconoscere, e queste parti sono ottenute dalla parola applicando l'omomorfismo.

Supponiamo che P accetti $h(w)$, allora abbiamo $(q_0, h(w), Z_0) \vdash_P^* (p, \epsilon, \gamma)$ con $p \in F$. Vediamo che P' accetta w . Supponiamo $w = a_1 a_2 \dots a_n$. Abbiamo $((q_0, \epsilon), w, Z_0) \vdash_{P'} ((q_0, h(a_1)), a_2 \dots a_n, Z_0)$, e dobbiamo ora usare le regole che consumano elementi dal buffer, quindi

$$((q_0, h(a_1)), a_2 \dots a_n, Z_0) \vdash_{P'}^* ((q', \epsilon), a_2 \dots a_n, \gamma_1)$$

che corrisponde a

$$(q_0, h(a_1)h(a_2 \dots a_n), Z_0) \vdash_P^* (q', h(a_2 \dots a_n), \gamma_1)$$

in P . Chiaramente si arriva a $((p, \epsilon), \epsilon, \gamma)$ con $p \in F$ se e solo se consumo tutta $h(w)$.

Quindi $\mathcal{L}_{PDA}(P') = h^{-1}(\mathcal{L})$.

Intersezione

Teorema 21 *I linguaggi context free non sono chiusi rispetto all'intersezione.*

Consideriamo $\mathcal{L}_1 = \{a^n b^n c^m \mid n, m \geq 1\}$ e $\mathcal{L}_2 = \{a^n b^m c^m \mid n, m \geq 1\}$.
Chiaramente sono entrambi context free ma la loro intersezione $\{a^n b^n c^n \mid n \geq 1\}$ non lo è.

Conseguenze: i linguaggi context free non sono chiusi rispetto alla complementazione e non sono chiusi rispetto alla differenza.

0.4 Pumping lemma

L'intuizione nel pumping lemma per i linguaggi regolari era: per parole sufficientemente lunghe l'automa a stati finiti deve ripassare per due volte nello stesso stato... la parola che ho visto dal primo al secondo attraversamento la posso pompare. Per il linguaggi context free ragiono sull'albero sintattico. Se nell'albero di una derivazione di una grammatica in forma normale di Chomsky c'è un cammino abbastanza lungo, allora devo vedere almeno due volte lo stesso simbolo non terminale. Le parole derivate derivate da quel simbolo le posso pompare.

Vediamo prima la relazione tra $|w|$ e l'albero sintattico.

Teorema 22 *Sia $G = (\mathbf{V}, \Sigma, \mathcal{P}, S)$ una grammatica in forma normale di Chomsky e sia $w \in \mathcal{L}(G)$. Se la lunghezza del cammino più lungo del suo albero di derivazione è n allora $|w| \leq 2^{n-1}$.*

Per induzione su n .

($n = 1$). Allora la derivazione corrispondente usa una produzione che riscrive una variabile in un terminale, quindi $|w| = 1$ oppure è la parola vuota. Chiaramente $|w| \leq 2^0 = 1$.

Sia vero per n , vediamo per $n + 1$, con $n \geq 1$. Chiaramente la prima produzione applicata è $A \rightarrow BC$ e per ipotesi induttiva $B \xRightarrow{*} w_1$ e $C \xRightarrow{*} w_2$ sono tali che nei sottoalberi con radice B e C il cammino più lungo è lungo al più n e quindi $|w_1| \leq 2^{n-1}$ e $|w_2| \leq 2^{n-1}$. Ora $w = w_1 w_2$ quindi $|w| = |w_1| + |w_2|$ e $|w| \leq 2^{n-1} + 2^{n-1} = 2^n$.

Teorema 23 *Sia $\mathcal{L}$ un linguaggio context free. Allora esiste una costante n tale che se $z \in \mathcal{L}$ e $|z| \geq n$ allora z può essere scomposta in cinque parti $z = uvwxy$ tali che*

1. $|vwx| \leq n$,
2. $vx \neq \emptyset$,
3. $\forall i \geq 0$ la parola $uv^i w x^i y \in \mathcal{L}$.

Partiamo da una grammatica in forma normale di Chomsky (se il linguaggio fosse vuoto oppure $\{\epsilon\}$, la dimostrazione è ovvia).

Supponiamo $\mathbf{V}$ contenga m variabili. Allora per una parola sufficientemente lunga (diciamo più di 2^m) vale che l'albero sintattico ha un cammino lungo almeno $m+1$ (sappiamo che $|z| \leq 2^{(m+1)-1}$). Siano $A_1, A_2, A_3, \dots, A_m, A_{m+1}, a$ le etichette dei nodi in questo cammino. Chiaramente un simbolo di variabile deve essere ripetuto due volte, supponiamo $A_h = A_k$ con $h < k$.

Costruiamo $S \xRightarrow{*}_{lm} uA_h\gamma B_1$, dove $S \rightarrow A_2B_1$ è la prima produzione applicata. Da A_h deriviamo $vA_k\gamma'$ e continuando otteniamo $vwA_{k+1}\gamma''$. Ora A_{k+1} genera una parola x e γ'' la parola y_1 . Infine da B_1 deriviamo y_2 . Osserviamo che $vw x$ viene generata dall'albero che ha radice A_h .

Se mettiamo assieme tutto abbiamo che la derivazione è:

$$S \xRightarrow{*}_{lm} uA_h\delta \xRightarrow{*}_{lm} uvA_k\delta' \xRightarrow{*}_{lm} uvwA_{k+1}\delta'' \xRightarrow{*}_{lm} uvwx\delta''' \xRightarrow{*}_{lm} uvwxy$$

Abbiamo $S \xRightarrow{*}_{lm} uA_h\delta \xRightarrow{*}_{lm} uvA_k\delta' \xRightarrow{*}_{lm} uvwA_{k+1}\delta'' \xRightarrow{*}_{lm} uvwx\delta''' \xRightarrow{*}_{lm} uvwxy$

Chiaramente $|uvw| \leq n$, e $vx \neq \epsilon$ dato che la grammatica non ha produzioni ϵ . Inoltre, dato che da A_h deriviamo $vw x$, ed il sottoalbero con radice A_h contiene $A_k = A_h$, possiamo anche derivare v^iwx^i per ogni $i \geq 0$.

Mostriamo che $\{0^n1^n2^n \mid n \geq 1\}$ non è context free. Assumiamo lo sia, esiste una costante m tale che possiamo scomporre ogni parola in cinque parti $uvwxy$ in modo tale $|vw x| \leq m$, $vx \neq \emptyset$ e che sia possibile *pompare* le v e x in modo che anche la parola uv^iwx^iy sia nel linguaggio. Prendiamo come parola $0^m1^m2^m$, allora qualsiasi scomposizione di $0^m1^m2^m$ in $uvwxy$ tale che $|vw x| \leq m$ deve essere tale che $vw x$ contiene o 0 ed 1, o 1 e 2. Comunque io divida $vw x$, dato che $vx \neq \epsilon$ vale che o v contiene solo 0 e x contiene solo 1, oppure v contiene solo 1 e x contiene solo 2. Comunque le pompi non sono in grado di avere parole che ancora stanno nel linguaggio, quindi la tesi.

Lo schema è sempre lo stesso: prendo una parola, devo mostrare che qualsiasi suddivisione che soddisfa le ipotesi del pumping lemma, porta alla falsificazione del fatto che la parola *pompata* appartiene al linguaggio.

Esercizi

1. considerate il linguaggio $\{0^n1^n \mid n \geq 1\}$. Trovate una grammatica in forma normale di Chomsky
2. considerate una grammatica in forma normale di Chomsky. Date un limite alla grandezza dell'albero di derivazione di una parola w lunga n
3. considerate la grammatica $S \rightarrow AB \mid CA, A \rightarrow a, B \rightarrow BC \mid AB$ e $C \rightarrow aB \mid b$. Trovate una grammatica equivalente senza simboli inutili.

4. considerate la grammatica $S \rightarrow ASB \mid \epsilon, A \rightarrow a \mid aAS, B \rightarrow SBS \mid A \mid bb$. Eliminate prima le produzioni ϵ , poi quelle unitarie, i simboli inutili e mettetela in forma normale di Chomsky. (chiaramente la nuova grammatica non genera ϵ : lo si può aggiungere come unica produzione dal simbolo di start S)
5. convertite in forma normale di Greibach la grammatica $S \rightarrow AA \mid 0, A \rightarrow SS \mid 1$
6. convertite in forma normale di Chomsky le grammatiche che sono state introdotte negli esercizi delle lezioni precedenti
7. mostrate che $\{a^i b^j c^k \mid i < j < k\}$ non è context free
8. mostrate che $\{a^n b^n c^m \mid m \leq n\}$ non è context free
9. mostrate che se $\mathcal{L}$ è context free, lo è anche $\mathcal{L}/a = \{w \mid wa \in \mathcal{L}\}$
10. considerate la sostituzione

$$s : \{0, 1\} \rightarrow \{\{w \in \{a, b\}^* \mid w = w^R\}, \{a^n b^{2n} \mid n \geq 0\}\}$$

Dite quale è la grammatica di $s(\{w \in \{0, 1\}^* \mid \text{il numero di 0 in } w \text{ è il doppio del numero di 1}\})$

11. mostrate che $\{0^n 1^{2n} \mid n \text{ pari}\}$ è context free